

SENTIMENT ANALYSIS USING ML

Harsha Deshpande	202007927
Kanchan Patil	202008246
Lizzen Camelo	202007935
Quisha Coutinho	202007945

Dissertation submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER ENGINEERING

OF GOA UNIVERSITY



Year 2024

COMPUTER ENGINEERING
PADRE CONCEICAO COLLEGE OF ENGINEERING
VERNA GOA – 403722

SENTIMENT ANALYSIS USING ML

Harsha Deshpande	202007927
Kanchan Patil	202008246
Lizzen Camelo	202007935
Quisha Coutinho	202007945

Dissertation submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER ENGINEERING

OF GOA UNIVERSITY



2024 Year

COMPUTER ENGINEERING
PADRE CONCEICAO COLLEGE OF ENGINEERING
VERNA GOA – 40372

**PADRE CONCEICAO COLLEGE OF ENGINEERING
VERNA GOA – 403722**



SENTIMENT ANALYSIS USING ML

Bona fide record of work done by

Harsha Deshpande	202007927
Kanchan Patil	202008246
Lizzen Camelo	202007935
Quisha Coutinho	202007945

Dissertation submitted in partial fulfillment of the requirements for the degree of Bachelor of engineering in Computer engineering under of Goa University

2024 Year

Approved By:

.....
Dr. Supriya Patil
Associate Professor

.....
Dr. Anusha Pai
Head of the Department

.....
Dr. Mahesh Parappagoudar
Principal

Examined By:

Examiner 1:
(Name / Signature / Date)

Examiner 2:
(Name / Signature / Date)

**PADRE CONCEICAO COLLEGE OF ENGINEERING
VERNA GOA – 403722**



STUDENT DECLARATION

We hereby declare that the project report titled "SENTIMENT ANALYSIS USING ML" submitted by us to Padre Conceição College of Engineering, Verna in partial fulfillment of the requirement for the award of the degree of Bachelor of Engineering in Computer Engineering under of Goa University is a record of bona fide project work carried out by us under the guidance of Dr. Supriya Patil.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Student Name

Student Signature with date

- | | | |
|----|------------------|-----------|
| 1) | Harsha Deshpande | 202007927 |
| 2) | Kanchan Patil | 202008246 |
| 3) | Lizzen Camelo | 202007935 |
| 4) | Quisha Coutinho | 202007945 |

CONTENTS

CHAPTER	Page No.
ACKNOWLEDGMENT	i
ABSTRACT.....	ii
LIST OF FIGURES	iii
LIST OF TABLES	v
LIST OF SYMBOLS, ABBREVIATIONS OR NOMENCLATURE.....	vi
1. INTRODUCTION	1
1.1 Purpose.....	1
1.2 Project Scope	1
1.3 Literature Survey	2
1.3.1 Overview.....	2
1.3.2 Basic Concepts of Sentiment Analysis	2
1.3.3 Research Paper Findings.....	3
1.3.4 Machine Learning Classifiers	8
1.3.4.1 Naïve Bayes Classifier.....	8
1.3.4.2 Support Vector Machine.....	9
1.3.4.3 Random Forest.....	10
1.3.4.4 Logistic Regression.....	11
1.3.5 Deep Learning Classifiers.....	12
1.3.5.1 Long-Short Term Memory.....	12
1.3.5.2 Bidirectional Encoder Representation from Transformers.....	13
1.3.5.3 BERT-BiLSTM Hybrid Model.....	14
2. OVERALL DESCRIPTION.....	15
2.1 Project Perspective.....	15
2.2 Project Functions	15
2.3 User Classes and Characteristics	15
2.4 Operating Environment.....	15
3. REQUIREMENTS.....	16
3.1 Functional Requirements	16
3.2 Nonfunctional Requirements	16
3.3 Hardware Requirements	16
3.4 Software Requirements.....	16
4. SYSTEM DESIGN.....	18
4.1 Research Frameworks.....	18

4.1.1 Data Preprocessing	18
4.1.2 Sentiment Analysis Process	20
4.2 Application Frameworks.....	20
4.2.1 System Flowchart	20
4.2.2 Application Architecture.....	22
5. EXPERIMENTATION AND RESULTS	23
5.1 Experiment Implementation	23
5.1.1 Preprocessing	23
5.1.2 Data Transformation	24
5.1.2.1 Term Frequency – Inverse Document Frequency.....	24
5.1.2.2 Count Vectorizer.....	24
5.1.3 Train-Test Ratio	24
5.1.4 Machine Learning Algorithms.....	24
5.1.4.1 Naïve Bayes Classifier.....	24
5.1.4.2 Support Vector Machine	26
5.1.4.3 Random Forest.....	27
5.1.4.4 Logistic Regression.....	28
5.1.5 Deep Learning Algorithms	29
5.1.5.1 LSTM Model	29
5.1.5.2 BERT Model.....	29
5.1.5.3 BERT-BiLSTM Hybrid Model.....	30
5.1.5 Evaluation Metrics	31
5.1 Results	31
6. DESIGN AND INTEGRATION.....	34
6.1 Design Concept	34
6.1.1 Color Scheme.....	34
6.1.2 Font Scheme	34
6.2 React Packages Used	35
6.3 API Endpoints	36
6.4 Dashboard Elements	37
6.5 Back-end Processing	38
6.5.1 Data Storage.....	38
6.5.4 Data Collection	38
6.5.4 Data Processing.....	39
6.5.4 Data Retrieval and Analysis	39
7. IMPLEMENTATION	40
7.1 Front-end Codes	40

7.2 Back-end Codes	44
8. OUTPUT	48
CONCLUSION	51
FUTURE SCOPE	52
REFERENCES.....	53

ACKNOWLEDGMENT

We take this opportunity to wholeheartedly thank those who were instrumental in helping us achieve our goals.

We are grateful to our respected Principal, **Dr. Mahesh B. Parappagoudar** and to our respected Head of Department of the Computer Department, **Dr. Anusha Pai** for granting us the opportunity and providing facilities, thereby allowing us to work on our project diligently and helping us through all the official work.

We sincerely express our gratitude to our internal project guide, **Dr. Supriya Patil**, Associate Professor, and our co-guides, **Ms. Cynara Marianne Silveira**, Assistant Professor, and **Ms. Ramita P. Karpe**, Assistant Professor, for their encouragement, guidance, and valuable help.

We would also like to thank the staff of the Department of Computer Engineering for their cooperation and support.

We are very grateful to our parents and certain well-wishers, who provided us with lots of support. We also thank all the members of the group for their cooperation, sincere hard work, and dedication, as well as our friends, who provided lots of moral support and extended their helping hand in times of need.

ABSTRACT

In the evolving landscape of e-commerce, understanding consumer sentiments towards products is necessary for businesses. The project was conducted in two phases- Research and Application Development. Research was conducted to study various machine learning (ML) and deep learning (DL) algorithms in the field of sentiment analysis. Using the knowledge gained from research, a comprehensive web application was developed to perform sentiment analysis on user-entered product reviews and provide sentiment analysis conducted on the product.

The Product Sentiment Analysis Application addressed the growing need for accurate and real-time understanding of consumer. The application utilized natural language processing (NLP) algorithms to analyze product reviews for expressions of sentiments, which improved decision making around the product and customer satisfaction in general.

A meticulous comparative study was carried out to explore various ML and DL algorithms, and determine their effectiveness in accurately classifying sentiments expressed in textual data. This phase of the project identified the combination of feature extraction methods and algorithms that delivered high accuracy in sentiment classification for the large dataset chosen.

The project then transitioned into the application development stage. The resulting web application enabled users to input product reviews and generated a dashboard that provided insights into the product based on the analyzed sentiments. This user-friendly interface allowed individuals, regardless of their familiarity with sentiment analysis, to gain valuable insights into consumer opinions.

In essence, this project seamlessly integrated research with practical application development, delivering a Product Sentiment Analysis Application that stood as a testament to the learnings from the research phase.

LIST OF FIGURES

Figure	Name	Page No.
Fig 1.1	Two classes segregated by SVM hyperplane	(9)
Fig. 1.2	Conceptual Framework of Random Forest Classifier	(10)
Fig. 1.3	The Sigmoid Curve	(11)
Fig. 1.4	LSTM Architecture	(12)
Fig. 1.5	BERT Architecture	(13)
Fig. 1.6	BERT-BiLSTM Hybrid Model	(14)
Fig. 4.1	Preprocessing without handling negation	(19)
Fig. 4.2	Preprocessing handling negation	(19)
Fig. 4.3	Sentiment Analysis Process	(20)
Fig 4.4	System Flowchart	(21)
Fig. 4.5	Application Architecture	(22)
Fig. 5.1	Dataset before Preprocessing	(23)
Fig. 5.2	Dataset after Preprocessing	(24)
Fig. 5.3	Snippet of result logs for NB	(31)
Fig. 5.4	Best Observed Accuracies	(32)
Fig. 5.5	Highest Accuracies by ML algorithms on Dataset 1	(32)
Fig. 5.6	Highest Accuracies by ML algorithms on Dataset 2	(32)
Fig. 5.7	Highest Accuracies by DL algorithms	(32)

Fig. 6.1	Color Palette for Application	(34)
Fig. 6.2	Font Style 1: Nanum Gothic Coding	(35)
Fig. 6.3	Font Style 2: Major Mono Display	(35)
Fig. 8.1	CSV file given as input on Home Page	(48)
Fig. 8.2	URL given as input on Home Page	(48)
Fig. 8.3	Dashboard	(49)
Fig. 8.4	Dashboard (contd.)	(50)

LIST OF TABLES

Table	Name	Page No.
Table 1.1	Summary of key findings from Research Papers	(3)
Table 5.1	Performance of TF-IDF and NB	(32)
Table 5.2	Performance of CV and NB	(32)
Table 5.3	Performance of TF-IDF and SVM	(32)
Table 5.4	Performance of CV and SVM	(32)
Table 5.5	Performance of TF-IDF and RF	(32)
Table 5.6	Performance of CV and RF	(32)
Table 5.7	Performance TF-IDF and LR	(32)
Table 5.8	Performance CV and LR	(32)
Table 5.9	Performance of LSTM Model	(32)
Table 5.10	Performance of BERT and BERT+BiLSTM Hybrid Model	(32)
Table 6.1	Summary of API Endpoints	(36)
Table 6.2	Review Table in Database	(38)

LIST OF ABBREVIATIONS

1.NLP:	Natural Language Processing
2.ML:	Machine Learning
3.DL:	Deep Learning
4.SVM:	Support Vector Machine
5.LSTM:	Long Short-Term Memory
6.NB:	Naïve Bayes
7.KNN:	K Nearest Neighbors
8.TF-IDF:	Term Frequency – Inverse Term Frequency
9.BERT:	Bidirectional Encoder Representation from Transformers
10. RNN:	Recurrent Neural Networks
11. Bi-LSTM:	Bi-Directional LSTM
12. URL:	Uniform Resource Locator
13. CSV:	Comma Separated Values
14. API:	Application Programming Interface

CHAPTER 1

INTRODUCTION

This innovative project combines insightful research with practical application development, resulting in a Product Sentiment Analysis Application. By harnessing advanced NLP algorithms, the application promises to encapsulate how businesses interpret and respond to consumer sentiments.

1.1 PURPOSE

The project serves a dual purpose, consisting of research and practical application development. Firstly, the project's research component delves into an extensive exploration of various Machine Learning and Deep Learning algorithms, analysing their performance in deciphering sentiments from textual data. This investigative process aims to unravel the nuanced dynamics of sentiment analysis, providing valuable insights into the adaptability and accuracy of different algorithms. By engaging in this research, the project advances the field's understanding and also equips the developers with a profound knowledge base.

Secondly, the Sentiment Analysis Application serves as a practical manifestation of the acquired knowledge, offering hands-on experience in creating an end-to-end application. The development phase transforms theoretical insights into a functional tool, incorporating NLP techniques and algorithmic advancements identified during the research. This practical application phase enables the developers to hone their skills in designing, implementing, and optimising a sophisticated sentiment analysis system.

1.2 PROJECT SCOPE

- The project scope encompasses a comprehensive exploration of sentiment analysis techniques through rigorous research, followed by the practical development of a robust Sentiment Analysis Application.
- A comparative study of various sentiment analysis algorithms is to be conducted and documented in the form of a research paper.
- An application is to be developed that integrates data collection (of various product reviews), processing of the reviews, and conducting sentiment analysis on the same to provide a dashboard as output that gives insights into the products to the end user.

1.3 LITERATURE SURVEY

1.3.1 Overview

Sentiment analysis, also known as opinion mining, is an NLP task aimed at determining the sentiment expressed in a piece of text. By analysing the subjective information present, it categorizes the sentiment as positive, negative, or neutral. This computational technique is widely used to extract insights from social media, customer reviews, and other textual data sources, offering valuable perspectives for businesses and researchers.

To understand the domain in depth, a literature survey involving an extensive examination of research papers was carried out, to refine the selection of classifiers for the project's implementation. The basic concepts involved in sentiment analysis were studied along with the algorithms, which were understood in-depth by implementing their hard-coded models.

1.3.2 Basic Concepts of Sentiment Analysis

1. **Data Preprocessing:** Data preprocessing involves cleaning and transforming raw text data to enhance the quality of input for machine learning models. This includes removing irrelevant characters, handling capitalization, tokenization, and eliminating stop words to prepare the text for analysis. Additionally, techniques like stemming or lemmatization may be applied to reduce words to their root form, aiding in feature extraction.

2. **Feature Extraction:** Feature extraction converts preprocessed text into numerical representations, allowing machine learning models to interpret sentiment. These features aim to represent the essence of the language used in sentiment expression.

The feature extraction methods used in this project are:

- a. **TF-IDF:** It is a numerical statistic that reflects the importance of a word in a document relative to a collection of documents (corpus). It consists of two components: Term Frequency (TF), which measures the frequency of a term in a document, and Inverse Document Frequency (IDF), which penalizes terms that are common across many documents. The product of TF and IDF yields a weight that represents the significance of a term within a specific document and across the entire corpus. It is widely used in sentiment analysis to convert a collection of text documents into numerical vectors.

- b. **Count Vectorizer:** It is a feature extraction technique that converts a collection of text documents into a matrix of token counts. Each row in the matrix corresponds to a document, and each column represents a unique word (or n-gram) in the entire corpus. The matrix entries indicate the frequency of each word in each document. It captures the occurrence of words in documents, providing a basis for understanding the distribution of terms and enabling the training of models to recognize patterns associated with sentiment.

- 3. Model Creation and Implementation:** Model creation involves selecting and training a suitable sentiment analysis model, such as Support Vector Machines (SVM), Naive Bayes, or deep learning models. The training phase exposes the model to labeled data, enabling it to learn patterns associated with different sentiments. Once trained, the best performing ML model is implemented by integrating it into an application, enabling real-time sentiment analysis on new text data.
- 4. Model Evaluation:** Model evaluation is crucial for assessing the performance of the sentiment analysis model. Metrics such as accuracy, precision, recall, F1 score, and MCC are employed to measure the model's effectiveness in correctly classifying sentiments. Cross-validation techniques, like k-fold validation, help ensure robust performance across diverse datasets. Regular evaluation, possibly in conjunction with monitoring tools, ensures the model's reliability in various scenarios and guides potential improvements or updates as needed.

1.3.3 Research Paper Findings

A thorough study was carried out by referring to several research papers to strengthen the knowledge base in the field of sentiment analysis which helped set the scope for the research paper. The findings from few of the research papers are summarized in Table 1.1.

Table 1.1 Summary of key findings from Research Papers

Title	Publisher	Dataset source and sample size	Train – Test ratio	Preprocessing techniques	Classification algorithms	Accuracy
[1] Sentiment analysis for Amazon reviews	Research Gate DOI: 10.13140/RG.2.2.13939.37920	Kaggle dataset consists of Amazon customer reviews (input text) and binary output labels (positive and negative). Number of reviews: 3,940,000	60000 random reviews between 4m reviews. 40,000 : 20,000	- Removal of HTML tags. - Filtering every symbol except for letters (a-z) and numbers (0-9). - Filtering out every word with length of 3 symbols or lower. For the classification with MNB and LSVM, a TF-IDF vectorizer is used. Whereas for the classification with LSTM, a method called tokenization is applied.	- Multinomial naive Bayesian (MNB) - Linear support vector machine (LSVM) - Long short-term Memory (LSTM) networks.	Best: LSTM 90% Accuracy 0.96 AUC 0.92 Precision 0.87 Recall 0.90 F1
[2] Sentiment Analysis of Mobile Phone Reviews using Classification algorithms	IEEE	82,815 mobile phone products reviews from Kaggle.	90%: 10%	1. lowercasing 2. removing links 3. removing numbers and punctuations 4. Remove emojis 5. Remove words with length <= 2 6. remove stop words 7. stemming 8. Lemmatization 9. Feature Extraction: Count Vectorizer	1. KNN 2. NB 3. SVM 4. Max Entropy 5. Decision Tree	1000: NB 73%. 5000: NB and Max Entropy 78% 10000: NB and Max Entropy 84.8% 20000: Max Entropy 82.7%

SENTIMENT ANALYSIS USING ML

[3]Comparison of Naive Bayes Algorithm and Support Vector Machine using PSO Feature Selection for Sentiment Analysis on E-Wallet Review	IOP Publishing	This study uses 500 positive and 500 negative reviews in .txt form for data training. Data collected through reviews on the Google Play store about OVO e-wallet.	1:1	1. Tokenization 2. Stop words removal 3. Stemming 4. Generate n-grams 5.Feature Selection: by PSO	1. NB 2. SVM	PSO + NB: 93.10%
[4]Feature Based Sentiment Analysis of Mobile Product Reviews using Machine Learning Techniques	IJATCSE	Mobile product review datasets (Kaggle website) composed of 200 Amazon reviews of 5 products in the mobile product domain		1)Tokenizing 2) POS-Tagging	Naive Bayesian, Bernoulli Naive Bayesian (BNB), K Nearest Neighbor (K-NN) and SVM	K-NN and BNB outperforms from other two algorithms in terms of accuracy.
[5]Sentiment analysis using product review data	Springer	Set of product reviews collected from amazon.com. 5.1 million of product reviews. review includes: reviewer ID; product ID; rating; time of the review; helpfulness; review text.	40:30:30	1)tokenization 2) part-of speech (POS) Taggers 3) Negation phrases identification 4) Sentiment score computation for sentiment tokens 5) The ground truth labels a) sentence level categorization b) review-level categorization	Naïve Bayesian, Random Forest, and Support Vector Machine.	<i>sentence-level categorization</i> : Random Forest <i>review-level categorization</i> : SVM model and the Naïve Bayesian model are identical in terms of their performances.

SENTIMENT ANALYSIS USING ML

[6]NLP Based Sentiment Analysis on Twitter Data Using Ensemble Classifiers	ICSCN	Data collected from Twitter using Twitter API 1.1 with user search key as keyword.		1) Stop Word removal 2) Repeated letters removal 3) Noise data removal 4) POS tagging 5) Synset finding 6) Word Sense Identification 7) Feature Vector Formation	Ensemble method, machine learning algorithms like SVM, Baseline, Maximum Entropy and Naive Bayes	It is observed that the ensemble method outperforms the traditional classification methods by about 3-5%. Among the ensemble methods Extremely Randomized Trees classification performs better than others.
[7]A survey on sentiment analysis methods, applications, and challenges	Springer	Data collected from the internet via web scraping		1)Terms frequency 2) Parts of Speech tagging 3) Negations 4) Bag of Words (BoW) 5) Word embedding	Naive Bayes (NB), Support vector machine (SVM), Logistic regression (LR), Decision tree (DT), K-nearest neighbors (KNN), Maximum entropy (ME), Hybrid approach (combination of ML & lexicon-based approaches)	NB and SVM algorithms perform better than the rest of the algorithms.
[8]COMPARISON BETWEEN MACHINE LEARNING ALGORITHMS USED FOR SENTIMENT ANALYSIS	IJARET	a dataset of movie reviews from IMDB website (www.imdb.com) is used to train and test the model.		1)Vectorization 2) Term Frequency (TF) 3) Inverse document frequency(IDF)	Logistic regression, Naïve Bayes	Naive Bayes produces better results and has high accuracy of around 95%

SENTIMENT ANALYSIS USING ML

[9]A review on sentiment analysis and emotion detection from text	Springer	Sem Eval, Stanford sentiment treebank (SST), ISEAR, Data crawled from various social media platform's posts, blogs, ecommerce sites		1.Tokenization 2. Stemming & lemmatization 3. N-gram method 4. TF-IDF	1)ML based approach: Naïve Bayes, support vector machine (SVM), decision trees. 2)Deep Learning-based Approach: CNN, LSTM, and Bi-LSTM	deep learning approach performs better than machine learning. RNN, especially the LSTM model, are prevalent in sentiment analysis
[10]Opinion Mining and Sentiment Analysis on Online Customer Review	2016 IEEE International Conference on Computational Intelligence and Computing Research	Taken from Amazon for 3 electronic products Apple iPhone 5S, Samsung J7 and Redmi Note 3.	N/A	Tokenization, Bag-of-Words (BoW), TF-IDF (Term Frequency-Inverse Document Frequency)	Logistic Regression, Naïve Bayes Text Classification, SentiWordNet	Naïve Bayes performed the best and had the Lowest PRECISION values For BOW preprocessing technique.
[11] Sentiment Analysis using Machine Learning Techniques on Python	IEEE	Amazon reviews Yelp reviews IMDB reviews Kaggle dataset Indian Airlines reviews	75-25	Inbuilt libraries- 're' and 'nltk' a) Replacement of single characters at the beginning of sentences by spaces(like I, A etc.). b) Converting all the text to lowercase. c) Removal of Stop words d) Creation of feature vector	-Logistic Regression -k - Nearest Neighbors -Support Vector Machine Classifier -Decision Tree Classifier -Random Forest Classifier -AdaBoost Classifier -Gaussian Naive Bayes -Bagging Classifier	Amazon - Bagging Classifier (AdaBoost) 76.40% Yelp - Random Forest Classifier 76% IMDB - Random Forest Classifier 76% Indian-Airlines - AdaBoost 59.6%
[12] Sentiment classification of online consumer reviews using word vector representations	Science-Direct	400,000 mobile phone reviews from Amazon.	80-20	Stemming, stop words removal, converting to lower case, punctuations removal and white space elimination, tokenization. Feature Extraction: word2vec models, TF-IDF scheme	Naïve Bayes, SVM, Logistic Regression and Random Forest using 10-folds cross validation.	CBOW_RF: 90.66% SG_RF: 90.21% Mean AUC (area under curves)for CBOW_RF: 0.94, SG_RF: 0.93

[13] Business reviews classification using sentiment analysis	IEEE	10000 business reviews dataset for experiment 1. Filtered Yelp Challenge Dataset with 1.34 million text reviews for experiment 2	80-20	EXP 1: Punctuations and spaces removed. Lowercasing. Feature extraction: 2 approaches 1) removing punctuations + removing stop words + applying stemming + handling negations and using unigrams in forming the feature vectors in bag of words representation. 2) removing stop words + applying stemming + POS + applying WSD and computing score using the sentiment opinion lexicon SentiWordNet EXP 2: remove stop words + apply stemming + handling negations + unigrams for feature vectors representation	EXP1: Using 3-fold cross validation. We adapt and apply four learning models: Multinomial Naïve Bayes, Support vector machines: Linear Support Vector Classification, Logistic regression and Stochastic Gradient Descent Classifier for sentiment analysis classification EXP 2: 3-fold cross validation for evaluating the systems accuracy	EXP 1: Using Stochastic Gradient Descent (SGD) Classifier (92.6%) using the first approach. EXP 2: 94.4 % with Linear SVC using the first approach. systems' accuracy could still be improved by exploring the usage of bigrams or trigram, word chunks, or even part-of-speech (POS) as features
[14] Effective Sentimental Analysis and Opinion Mining of Web Reviews Using Rule Based Classifiers	Springer India	Online book reviews and political reviews using JSoup Web Crawler		Stop Words Removal Stemming Parts of Speech Tagging	1.SentiWordNet 2.Decision table 3.Naïve Bayes 4.Conjunctive rule 5.JRipper 6.PART C4.5 7.RiDOR	Decision table Naïve Bayes and PART C4.5 (%) gave 99.14% for Political reviews Decision table Naïve Bayes gave 98.54% for Book reviews
[15] Generating Tips from Product Reviews	ACM	labeled dataset containing a total of 3,059 tips and 48,870 non-tip sentences	80-20	Data Annotation Rule-based Filtering	1. Naïve Bayes 2. LSTM 3. LSTM with Attention. 4. FastText. 5. BERT.	BERT outperforms with 58.05% recall for 80% precision.

1.3.4 Machine Learning Classifiers

1.3.4.1 Naïve Bayes Classifier

The Naïve Bayes classifier is a popular supervised machine learning algorithm used for classification tasks such as text classification. It belongs to the family of generative learning algorithms, which means that it models the distribution of inputs for a given class or category. This approach assumes that the features of the input data are conditionally independent given the class, allowing the algorithm to make predictions quickly and accurately.

In statistics, naive Bayes classifiers are considered as simple probabilistic classifiers that apply Bayes' theorem. This theorem is based on the probability of a hypothesis, given the data and some prior knowledge. The naive Bayes classifier assumes that all features in the input data are independent of each other, which is often not true in real-world scenarios. However, despite this simplifying assumption, the naive Bayes classifier is widely used because of its efficiency and good performance in many real-world applications.

Bayes' Theorem finds the probability of an event occurring given the probability of another event that has already occurred. Bayes' theorem is stated mathematically as the following equation, where A and B are events and $P(B) \neq 0$:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \quad (1.1)$$

Basically, we are trying to find probability of event A, given the event B is true. Event B is also termed as evidence.

- $P(A)$ is the priori of A (the prior probability, i.e. Probability of event before evidence is seen). The evidence is an attribute value of an unknown instance(here, it is event B).
- $P(B)$ is Marginal Probability: Probability of Evidence.
- $P(A|B)$ is a posteriori probability of B, i.e. probability of event after evidence is seen.
- $P(B|A)$ is Likelihood probability i.e. the likelihood that a hypothesis will come true based on the evidence.

Bayes' theorem states the following relationship, given class variable y and dependent feature vector x_1 through x_n :

$$P(y | x_1, \dots, x_n) = \frac{P(y)P(x_1, \dots, x_n | y)}{P(x_1, \dots, x_n)} \quad (1.2)$$

Using the naive conditional independence assumption that

$$P(x_i|y, x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n) = P(x_i|y), \quad (1.3)$$

for all i , this relationship is simplified to

$$P(y | x_1, \dots, x_n) = \frac{P(y) \prod_{i=1}^n P(x_i | y)}{P(x_1, \dots, x_n)} \quad (1.4)$$

Since $P(x_1, \dots, x_n)$ is constant given the input, we can use the following classification rule:

$$P(y | x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i | y) \Rightarrow \hat{y} = \operatorname{argmax}_y P(y) \prod_{i=1}^n P(x_i | y) \quad (1.5)$$

and we can use Maximum A Posteriori (MAP) estimation to estimate $P(y)$ and $P(x_i | y)$; the former is then the relative frequency of class y in the training set.

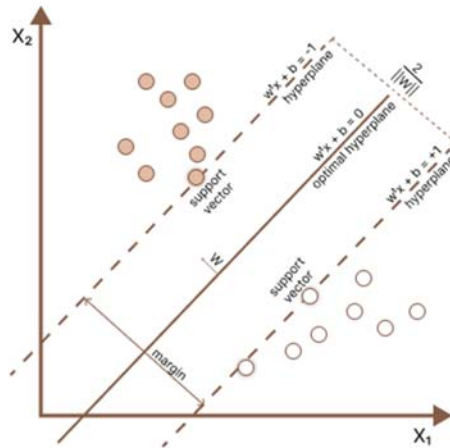
1.3.4.2 Support Vector Machine

The Support Vector Machine (SVM) is a supervised machine learning algorithm that excels in both classification and regression tasks. SVM has gained widespread acclaim for its efficacy in handling complex, high-dimensional datasets and achieving robust generalization performance. The primary goal of SVM is to identify a hyperplane in an N-dimensional space, where N represents the number of features, that distinctly separates data points of different classes while maximizing the margin between them. This margin, defined as the distance between the hyperplane and the nearest data point from each class, is crucial for enhancing the algorithm's ability to generalize to unseen data. Support vectors, the data points closest to the hyperplane, play a pivotal role in determining the position and orientation of the hyperplane.

The mathematical formulation of SVM involves optimizing an objective function, considering both linearly separable cases and non-linearly separable cases through the introduction of a regularization parameter. In cases where the data is not perfectly separable, SVM allows for a soft margin that tolerates some misclassifications. The trade-off between maximizing margin and minimizing misclassifications is controlled by a parameter called C.

Furthermore, SVM can handle non-linearly separable data using the kernel trick, which involves transforming input features into a higher-dimensional space. This algorithm finds applications in diverse domains such as image recognition, text classification, bioinformatics, and finance, owing to its versatility and effectiveness in complex scenarios. In essence, SVM stands as a robust and versatile algorithm, showcasing its effectiveness in real-world applications through its capability to maximize the margin between classes.

Fig 1.1 Two classes segregated by SVM hyperplane



The equations of the margin lines are as follows:

1. Equation of the hyperplane : $w^T x + b = 0$
2. Equation of the margin line in the positive area: $w^T x + b = +1$
3. Equation of the margin line in the negative area: $w^T x + b = -1$

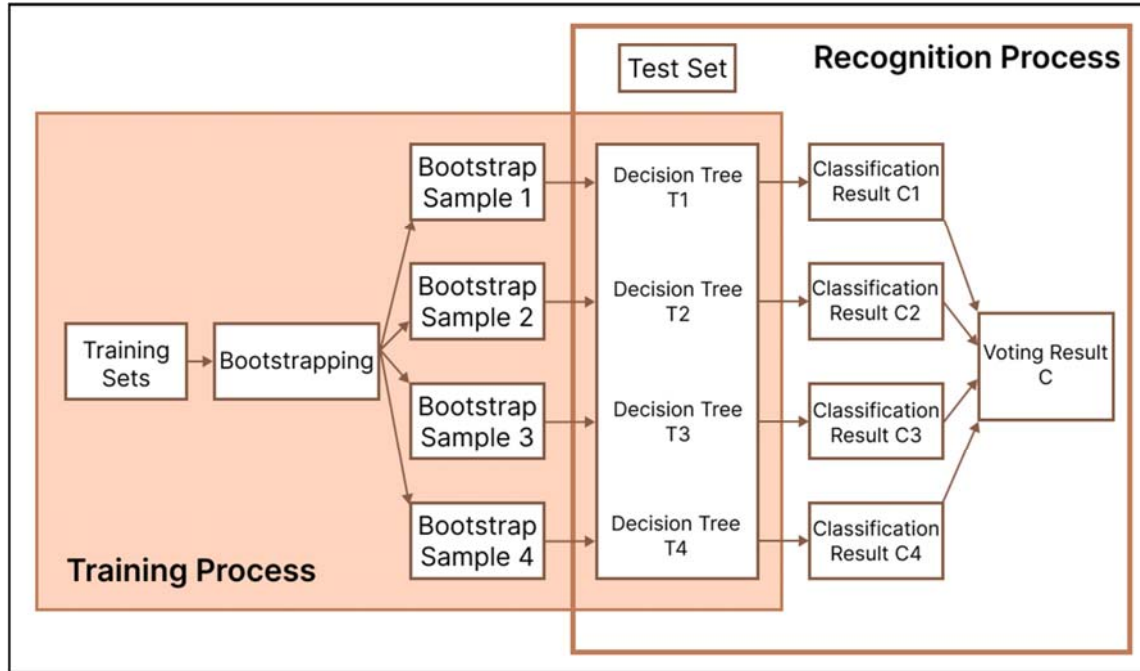
From the equation shown above, we need to maximize this function in order to find the best margin lines which guarantee that no data points are misclassified. The model should find the values of w and b that maximize the following function.

$$(w^*, b^*) \max \frac{2}{\|w^T\|} y_i \begin{cases} +1 & \text{where } w^T x_i + b \geq 1 \\ -1 & \text{where } w^T x_i + b \leq -1 \end{cases} \quad (1.6)$$

1.3.4.3 Random Forest

Random Forest generates an ensemble of decision trees. Input a test sample to the new classifier and the class label of this sample can be decided based on the voting results from each single classification. The entire process of classification based on the random forest is shown in Fig. 1.2.

Fig. 1.2 Conceptual Framework of Random Forest Classifier



To achieve diversity among base decision trees, a randomization approach which works well with bagging or random subspace methods is used. To generate each single tree in Random Forest, following steps were followed: If the number of records in the training set is N , then N records are sampled at random but with replacement, from the original data, this is a bootstrap sample. This sample will be the training set for growing the tree. If there are M input variables, a number $m \ll M$ is selected such that at each node, m variables are selected at random out of M and the best split on these m attributes is used to split the node. The value of m is held constant during forest growing. Each tree is grown to the largest extent possible. There is no pruning.

In this way, multiple trees are induced in the forest; the number of trees is pre-decided by the parameter N tree. The number of variables (m) selected at each node is also referred to as k in the literature. The depth of the tree can be controlled by a parameter node size (i.e. number of instances in the leaf node) which is usually set to one.

Once the forest is trained or built as explained above, to classify a new instance, it is run across all the trees grown in the forest. Each tree gives classification for the new instance which is recorded as a vote. The votes from all trees are combined and the class for which maximum votes are counted (majority voting) is declared as classification of the new instance.

1.3.4.4 Logistic Regression

Logistic Regression is a statistical method used for binary classification problems. It predicts the probability of an instance, a dependent variable belonging to a particular class (usually 0 or 1) based on one or more independent variables. Despite its name, it is used for classification, not regression.

The primary goal of Logistic Regression is to model the relationship between one or more independent variables and the probability of a binary outcome. It establishes a linear decision boundary that separates the feature space into two regions corresponding to the two classes.

Logistic Regression is widely used in machine learning for tasks where the dependent variable is categorical and represents two classes. Some of the applications include, finance (credit scoring), healthcare (disease prediction), marketing (customer churn), and natural language processing (sentiment analysis).

The Sigmoid Function: The logistic regression model uses the logistic function or sigmoid function to model the probability:

$$P(Y = 1) = \frac{1}{1 + e^{-(b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n)}} \quad (1.7)$$

- $P(Y = 1)$ is the probability of the dependent variable Y being 1.
- e is the base of the natural logarithm.
- $b_0, b_1, b_2, \dots, b_n$ are the coefficients of the model.
- $x_1, x_2, \dots, x_n$ are the independent variables or the input features.

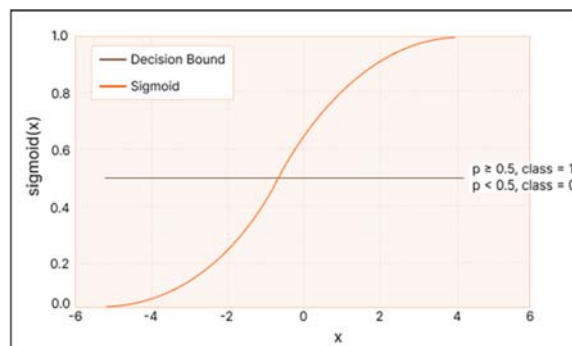
Coefficients ($b_0, b_1, b_2, \dots, b_n$): The logistic regression algorithm adjusts the coefficients during the training phase to minimize the difference between the predicted probabilities and the actual labels.

The Sigmoid function is helpful in logistic regression because it transforms the output of the linear combination of features into a probability scale (ranging from 0 to 1), allowing for binary classification decisions.

The decision boundary, where the probability is typically set to 0.5, separates the two classes. If the predicted probability is greater than the threshold, the instance is classified as the positive class; otherwise, it's classified as the negative class.

In context with sentiment analysis if this probability is greater than a certain threshold (e.g., 0.5), the model predicts a positive sentiment otherwise, it predicts a negative sentiment.

Fig. 1.3 The Sigmoid Curve



1.3.5 Deep Learning Classifiers

1.3.5.1 Long Short-Term Memory (LSTM)

Long Short-Term Memory is an advanced form of Recurrent Neural Network (RNN). LSTM effectively addresses RNN's limitations, particularly the vanishing gradient problem, making it superior for remembering long-term dependencies. This neural network integrates complex algorithms and gated cells, allowing it to retain and manipulate memory effectively, which is pivotal for applications like video processing and reading comprehension.

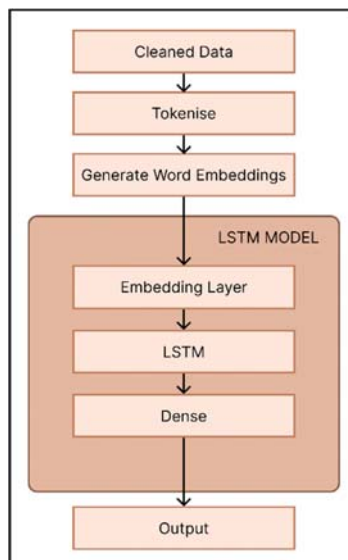
LSTM is a cell that consists of 3 gates. A forget gate, input gate, and output gate. Forget gate is responsible for deciding what information should be removed from the cell state. The input gate considers the current input and the hidden state of the previous time stamp. The input gate is used to update the cell state value. The output gate returns the hidden state for the next time stamp.

The cell has two states- cell state and hidden state. The intuition behind LSTM is that the cell and hidden states carry the previous information and pass it on to future time steps. The cell state is aggregated with all the past data information and is the long-term information retainer. The hidden state carries the output of the last cell, i.e. short-term memory.

The forget gate and input gate update the cell state. The cell state of the previous state is multiplied by the output of the forget gate. The output of this state is then summed with the output of the input gate. This value is then used to calculate hidden state in the output gate.

The LSTM architecture makes use of LSTM cell. The sequence of LSTM cells in each layer is fed with the output of the last cell. This enables the cell to get the previous inputs and sequence information. A cyclic set of steps happens in each LSTM cell- first, the Forget gate is computed. Then the Input gate value is computed followed by updating the cell state using the above two outputs. Finally, the output(hidden state) is computed using the output gate. The LSTM architecture implemented for experimentation is given by Fig. 1.4.

Fig. 1.4 LSTM Architecture



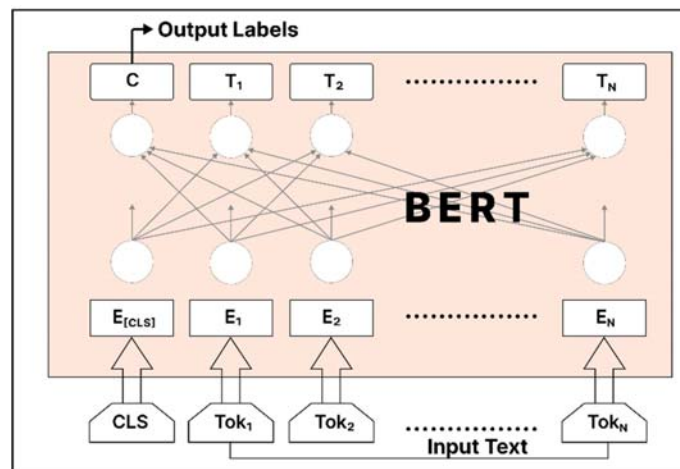
1.3.5.2 Bidirectional Encoder Representation Transformers (BERT)

BERT is designed to pretrain deep bidirectional representations from unlabelled text by jointly conditioning on both left and right context in all layers. There are two steps in our framework: pre-training and fine-tuning. During pre-training, the model is trained on unlabelled data over different pre-training tasks. For fine tuning, the BERT model is first initialized with the pre-trained parameters, and all of the parameters are fine-tuned using labelled data from the downstream tasks. Each downstream task has separate fine-tuned models, even though they are initialized with the same pre-trained parameters. A distinctive feature of BERT is its unified architecture across different tasks. There is minimal difference between the pre-trained architecture and the final downstream architecture.

BERT alleviates unidirectionality constraint of transformers by using a “masked language model” (MLM) pre-training objective. The masked language model randomly masks some of the tokens from the input, and the objective is to predict the original vocabulary id of the masked word based only on its context. Unlike left-to-right language model pre-training, the MLM objective enables the representation to fuse the left and the right context, which allows us to pretrain a deep bidirectional Transformer. In addition to the masked language model, we also use a “next sentence prediction” task that jointly pretrains text-pair representations. NSP (Next Sentence Prediction) is used to help BERT learn about relationships between sentences by predicting if a given sentence follows the previous sentence or not.

BERT working begins with tokenizing input into individual tokens, including the special [CLS] token which is the output representation used for classification tasks. The tokens are converted to embeddings that are a combination of token embeddings, segment embeddings, and position embeddings. The core of BERT consists of multiple transformer encoder layers. Each encoder layer applies self-attention mechanisms to capture the contextual relationships between tokens in a bidirectional manner (considering both the left and right context). The arrows within the BERT box represent self-attention mechanisms. Each token attends to every other token in the sequence, allowing the model to understand the context of each token comprehensively. The final hidden states from BERT are used to produce output labels for various NLP tasks. The [CLS] token's representation i.e. C is typically used for classification.

Fig. 1.5 BERT Architecture

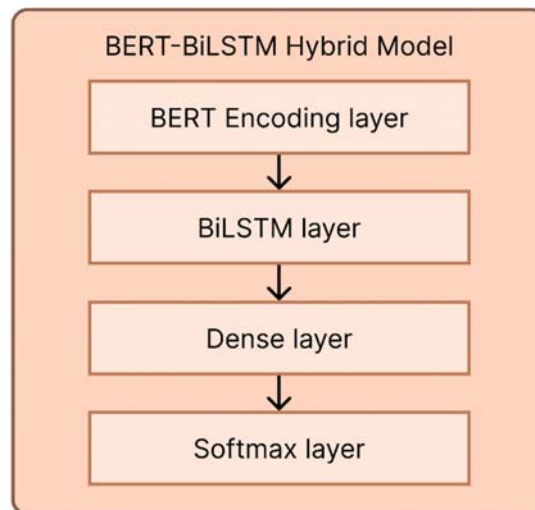


1.3.5.3 BERT-BiLSTM Hybrid Model

Bi-Directional LSTM or BiLSTM is an enhancement of traditional LSTM Architecture. In this method, there are two parallel LSTM Networks where one moves the data forward, while the other is moves it backward. In the scope of this project, a hybrid model is implemented wherein a pretrained BERT model is stacked with a bidirectional LSTM model to test if the accuracy of the BERT Model could have been increased.

BERT was be responsible for the data preprocessing and feature extraction. The output was given to the LSTM model to carry out the classification. BERT helps in mapping tokens into a meaningful embedding space which are then fed into the BiLSTM to capture the salient features. BERT provides powerful pre-trained word representations and captures contextual relationships. BiLSTM helps capture sequential dependencies within the sentence. The final classification layer predicts the most likely class label for the input sentence based on the learned features. Fig. 1.5 shows the architecture of the hybrid implementation.

Fig. 1.6 BERT-BiLSTM Hybrid Model



CHAPTER 2

OVERALL DESCRIPTION

This chapter covers the conceptualization, purpose, and pivotal functions of the Sentiment Analysis Application.

2.1 PROJECT PERSPECTIVE

The application will conduct sentiment analysis for the products aligned to a specific business to provide better accuracy with the benefits of being efficient, scalable, and easily maintainable compared to generic solutions. This project gives an edge over the existing projects as a thorough understanding of the domain area has been generated through the research phase, which is effectively translated into the application to serve its purpose.

2.2 PROJECT FUNCTIONS

1. Take the product URL from user to web-scrap reviews. Or, take reviews in CSV format.
2. Send the data through the API for preprocessing and model training after choosing an appropriate model based on the data provided.
3. Display insights generated by the sentiment analysis model to user in form of a dashboard.
4. Provide functionality to download the insights shown on the dashboard.

2.3 USER CLASSES AND CHARACTERISTICS

The main user classes of this project are listed as follows:

1. **Small Business Owners:** Owing to limited resources, they seek cost-effective solutions for understanding and responding to customer sentiments of their products being sold.
2. **Customers:** A customer can get insights on the product in a visualized format.
3. **Business Analysts:** This project can help them in their strategic decision-making by learning upon the trends shown by the product's insights.

2.4 OPERATING ENVIRONMENT

1. **Database Management System (DBMS):** A robust DBMS (Django's ORM with the default SQLite backend) is required to store and manage the dataset of product reviews, preprocessed data and sentiment predictions.
2. **Programming Languages:** ReactJS is used for front-end and Django for back-end.
3. **Hardware Requirements:** Client has no hardware requirements. The server end must be well-equipped with storage, compute etc.
4. **Network Requirements:** A stable internet connection needed for real-time data access.

CHAPTER 3

REQUIREMENTS

This segment looks into the requirements for building a web-based application for conducting sentiment analysis. The expectations of the application and the components would be required in developing and hosting the application are identified.

3.1 FUNCTIONAL REQUIREMENTS

1. The system must be able to web scrape data from the Web on the basis of the product URL provided as input. The system must also be able to accept a CSV file as input.
2. The system must be capable of processing and analysing textual data in real-time, providing instantaneous sentiment classification for timely decision-making.
3. The application should offer users the ability to view dashboards and download reports.

3.2 NON-FUNCTIONAL REQUIREMENTS

1. **Performance:** The application should process large datasets efficiently, with minimal latency.
2. **Scalability:** The system should be designed to handle increasing volumes of textual data and user requests without compromising performance or response times.
3. **Usability and Accessibility:** The user interface should be designed for ease of use and accessibility, catering to users with varying levels of technical expertise and accommodating accessibility standards.

3.3 HARDWARE REQUIREMENTS

1. **RAM:** Sufficient RAM is critical for handling large datasets and running memory-intensive processes during model training.
2. **GPU:** A dedicated GPU accelerates model training processes.
3. **Storage:** Ample storage space, is essential for storing large datasets, model weights, and intermediate results during the modeling phase.

3.4 SOFTWARE REQUIREMENTS

1. **Jupyter Notebook:** Jupyter Notebook is an open-source interactive web application that allows users to create and share documents containing live code, equations, visualizations, and narrative text. Widely used in data science and machine learning, it supports multiple programming languages, including Python.

2. **Colab:** Google Colab is a free, cloud-based platform provided by Google that allows users to write and execute Python code in a collaborative environment. It provides access to GPUs, making it popular for machine learning tasks, and allows seamless sharing of Jupyter Notebooks.
3. **Kaggle:** Kaggle is a leading online platform for data science and machine learning, offering a collaborative environment where users can access datasets, build models, and participate in competitions. It provides free GPU access and integrated Jupyter Notebook. Users can also explore various public datasets.
4. **Python Libraries:** A comprehensive set of tools for data preprocessing, feature extraction, model evaluation, and visualization in the context of sentiment analysis using Python which includes:
 - a. **NLTK (Natural Language Toolkit):** Used for NLP tasks, including tokenization, stop word removal, and stemming.
 - b. **NumPy and Pandas:** NumPy is used for numerical operations, and Pandas is employed for data processing and manipulation.
 - c. **Scikit-Learn:** It combines powerful model selection and feature extraction tools, facilitating seamless training and evaluation of machine learning models and a suite of evaluation metrics.
 - d. **PyTorch:** It is a flexible deep learning framework with a dynamic computational graph. It has a rich ecosystem of supporting libraries for various tasks and supports hardware acceleration for efficient training
 - e. **Transformers:** It provides pre-trained models excelling in NLP tasks. It offers state-of-the-art performance for NLP problems and enables fine-tuning pre-trained models for specific needs.
 - f. **Keras:** It is a user-friendly API built on top of other frameworks that simplifies deep learning model building and training. It is ideal for rapid prototyping and experimentation and integrates seamlessly with other deep learning libraries.
5. **Visual Studio Code:** Visual Studio Code (VS Code) is a free and open-source code editor developed by Microsoft. It supports various programming languages and features a customizable interface, integrated version control, and a rich extension ecosystem. It is widely used for web development, Python scripting, and general-purpose coding.
6. **Web Development Framework:** React is a JavaScript library for building user interfaces, particularly single-page applications with dynamic components. Django is a full-featured Python web framework that offers a comprehensive set of tools for building complex web applications, streamlining development from the ground up.
7. **GitHub:** GitHub is a web-based platform for version control and collaborative software development. It facilitates code hosting, version tracking, collaboration through pull requests, and project management. Developers use GitHub to share and contribute to open-source projects, manage code repositories, and collaborate with distributed teams.
8. **Google Sheets:** Google Sheets is a web-based spreadsheet application offered by Google as part of its Google Drive suite. It allows users to create, edit, and share spreadsheets online while collaborating with others in real-time.

CHAPTER 4

SYSTEM DESIGN

This chapter covers the data preprocessing flowcharts, the research framework, and the application workflow, providing a detailed look at how the system works.

4.1 RESEARCH FRAMEWORKS

4.1.1 Data Preprocessing

In the data preprocessing phase, the raw dataset is cleaned by performing several steps: dropping unnecessary columns and neutral reviews, removing missing values, resetting the index, and generating sentiment for each review.

The dataset is then processed through a clean reviews function, which includes:

1. **Lowercasing:** Converting all letters in the text to lowercase ensures uniformity and reduces the dimensionality of the text data in sentiment analysis.
2. **Tokenization:** Breaking down the text into individual tokens, such as words or phrases, facilitates subsequent analysis and feature extraction.
3. **Stemming:** Reducing words to their root or base form by removing suffixes, capturing the core meaning of words, and aiding in feature extraction while reducing dimensionality.
4. **Lemmatization:** Reducing words to their base or dictionary form (lemma). Unlike stemming, lemmatization considers the word's context and provides a more accurate representation of the original word in sentiment analysis.

Fig. 4.1 shows all steps followed for preprocessing without handling sentence negation in reviews, while Fig. 4.2 shows proposed steps to handle negation in reviews that implemented a small variation of the clean reviews function as opposed to the first method.

The `clean_reviews` function helped in carrying out lowercasing, tokenization, stemming and lemmatization of the dataset. The first dataset generated using the following `clean_reviews` function did not handle sentence negation. Whereas, the second dataset generated handled sentence negation.

The implementation to obtain the second dataset differed by two steps. First, the stop-words list generated had negative words like “not”, “couldn’t”, etc. dropped. Second, lemmatization was not carried out while implementing the `clean_reviews` function.

Fig. 4.1 Preprocessing without handling negation

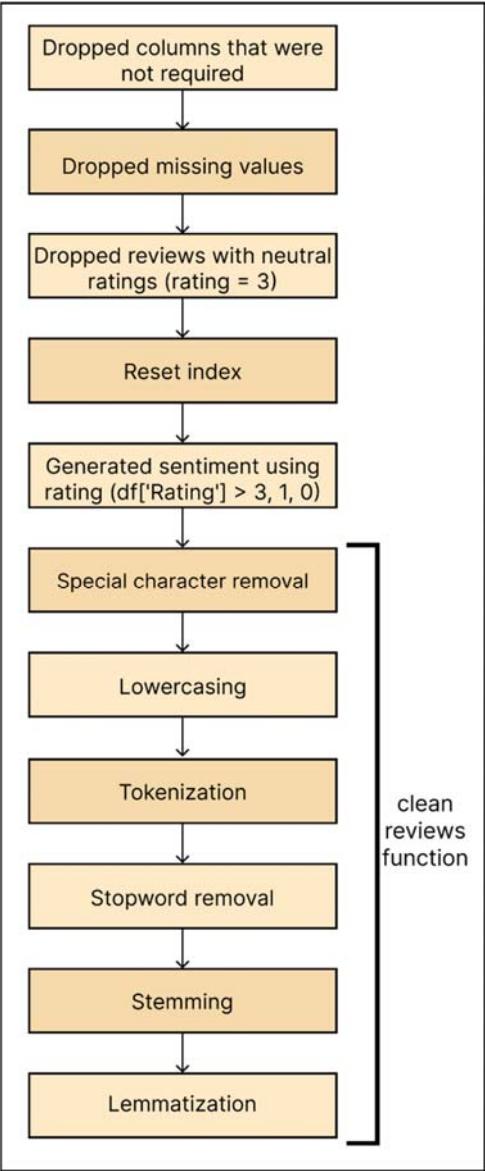
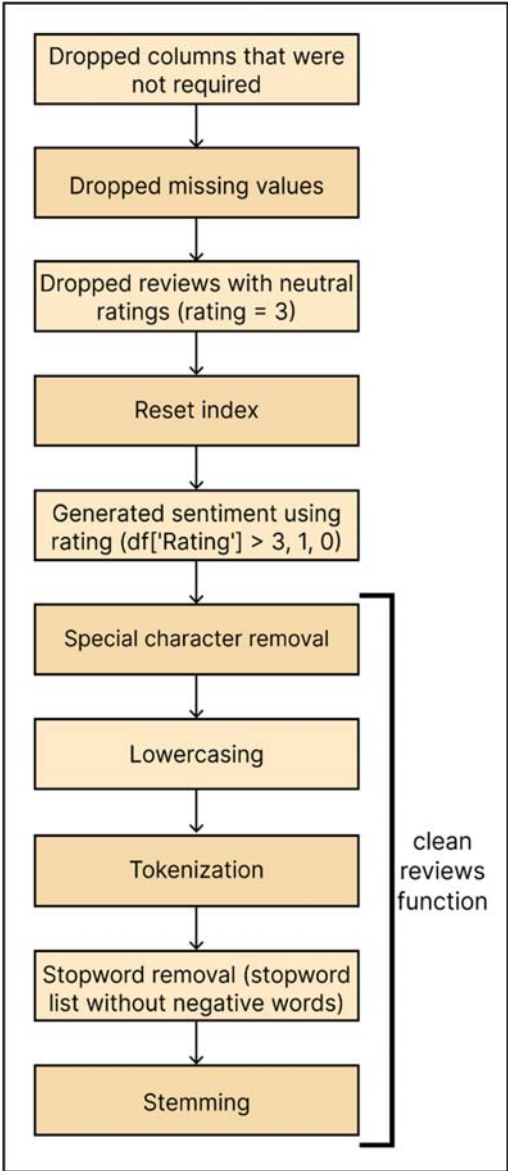


Fig. 4.2 Preprocessing handling negation

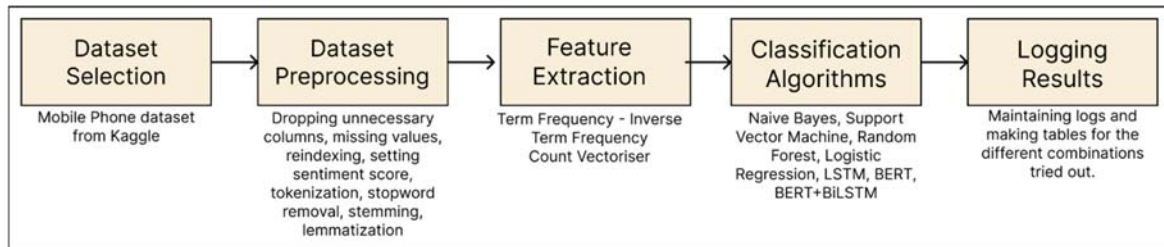


4.1.2 Sentiment Analysis Process

The experimentation phase involved exploring various combinations of feature extraction techniques and machine learning algorithms. The two chosen feature extraction techniques were combined with four selected machine learning algorithms. The hyperparameters of both the classifiers and the feature extractors were adjusted to optimize performance. Additionally, the experimentation process included studying deep learning algorithms at similar levels.

The combinations of feature extraction techniques and machine/deep learning algorithms were tested on the two cleaned datasets produced from the two approaches to preprocessing as discussed above. BERT and BERT-BiLSTM hybrid models, however, were processed on the raw dataset, as BERT has its own tokenizers to help with the data preprocessing.

Fig. 4.3 Sentiment Analysis Process



4.2 APPLICATION FRAMEWORKS

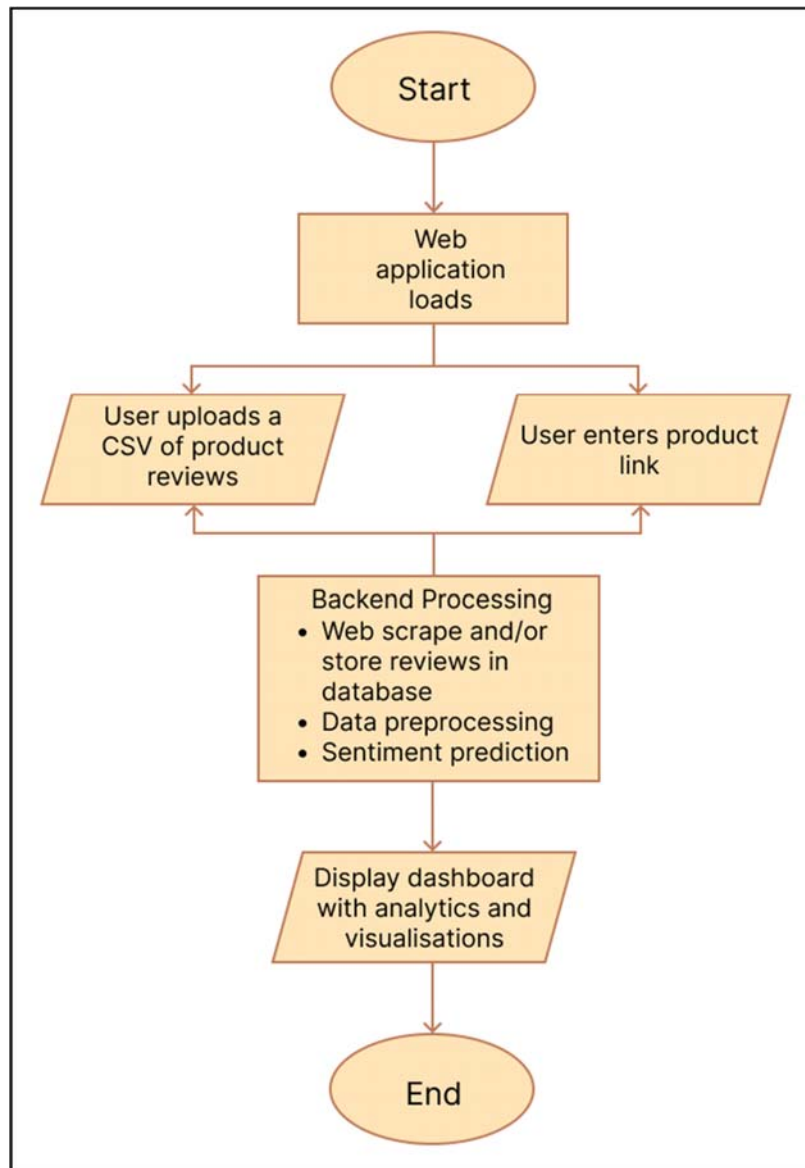
4.2.1 System Flowchart

The workflow of the application is described by Fig 4.4. The sequence of processes that occur when navigating through the application is described as follows:

1. The process begins when a user interacts with the web application by either uploading a CSV file containing product reviews or entering a link to an Amazon product page.
2. If the user uploads a CSV file, the application reads the file, parses the review data, and stores each review in a database. If the user provides an Amazon product link, the application initiates a web scraping process to retrieve the review data from Amazon. This involves logging into Amazon, navigating to the product page, fetching the HTML content, and extracting the review information. The extracted reviews are then stored in Django's database.
3. Once the reviews are collected, they undergo preprocessing to prepare the text data for analysis. This preprocessing includes converting the text to lowercase, removing non-alphabetical characters, tokenizing the text, removing stop words, and applying stemming to reduce words to their base forms. This cleaned text data is then used to predict the sentiment of each review.

4. The sentiment prediction process involves using a trained deep learning model to analyse the cleaned review texts and determine their sentiment as positive or negative. The predicted sentiments, along with their probabilities, are stored in the database.
5. To provide visual insights into the sentiment analysis, the application generates word clouds for both positive and negative reviews. The application also includes several features to display sentiment analysis results using chart visualisations like the line graphs, pie charts, bar charts, polar area charts, etc. These results are sent to the frontend through various API endpoints and visualisations are presented on the dashboard. The insights to help users understand the sentiment trends and key aspects mentioned in the reviews.

Fig. 4.4 System Flowchart

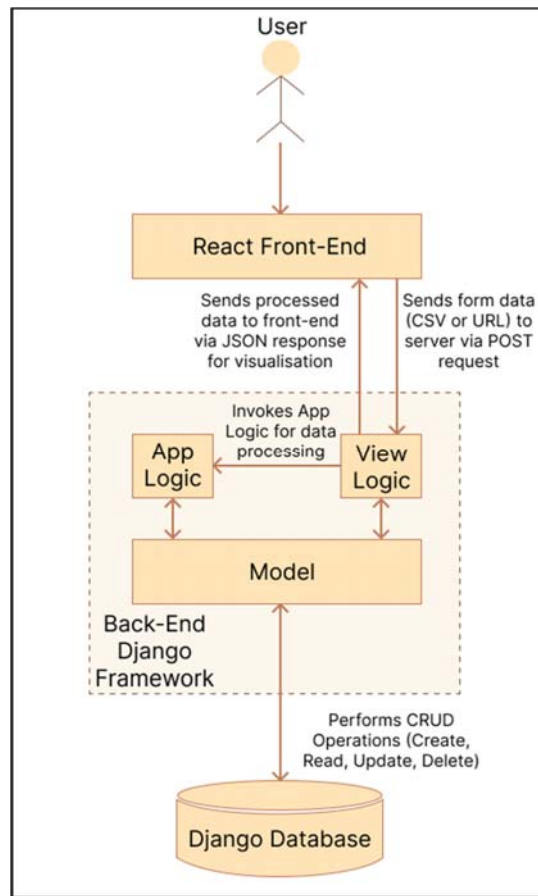


4.2.2 Application Architecture

Fig. 4.5 shows the architecture of the application. The different components utilized are described below.

1. **User:** The user represents the end-users who interact with the web application. These users can be individuals or entities who access the application's frontend through a web browser.
2. **Front-End (React):** The front-end application built with React sends form data (CSV or URL) to the Django backend via a 'fetch' call using the POST method.
3. **Django Back-End Framework:** The back-end receives the form data and routes the request to the appropriate view logic which implements the API.
4. **View Logic:** It processes the incoming HTTP requests. It invokes the application logic to perform tasks like data processing or database operations, and prepares the response to be sent back to the frontend for dashboard visualization.
5. **App Logic:** It encapsulates the core business rules of the application and processes data received from views. It performs operations on the database, such as querying, updating, and saving data.
6. **Database:** It stores the application's data. The app logic interacts with the database through Django's ORM (Object-Relational Mapping).

Fig. 4.5 Application Architecture



CHAPTER 5

EXPERIMENTATION AND RESULTS

In this segment the code snippets for the various processes that were involved in carrying out the sentiment analysis on the chosen dataset are mentioned along with the results of the experimentation process.

5.1 EXPERIMENT IMPLEMENTATION

5.1.1 Preprocessing

```
Cstopwords=set(stopwords.words('english')+list(punctuation))
from nltk.stem import WordNetLemmatizer
lemma=WordNetLemmatizer()

def clean_reviews(review_column):
    review_corpus=[]
    for i in range(0,len(review_column)):
        review=review_column[i]
        #replacing special characters
        review=re.sub('[^a-zA-Z]', ' ',review)
        #lowercasing
        review=str(review).lower()
        #tokenizing
        review=word_tokenize(review)
        #stemming
        review=[stemmer.stem(w) for w in review if w not in Cstopwords]
        #lemmatization
        review=[lemma.lemmatize(w) for w in review ]
        review=' '.join(review)
        review_corpus.append(review)
    return review_corpus
```

Fig. 5.1 Dataset before Preprocessing

	Product Name	Brand Name	Price	Rating	Reviews	Review Votes
0	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99	5	I feel so LUCKY to have found this used (phone...	1.0
1	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99	4	nice phone, nice up grade from my pantach revu...	0.0
2	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99	5	Very pleased	0.0
3	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99	4	It works good but it goes slow sometimes but i...	0.0
4	"CLEAR CLEAN ESN" Sprint EPIC 4G Galaxy SPH-D7...	Samsung	199.99	4	Great phone to replace my lost phone. The only...	0.0

Fig. 5.2 Dataset after Preprocessing

	Reviews	Rating	sentiment	clean_review
0	I feel so LUCKY to have found this used (phone...	5	1	feel lucky found use phone u use hard phone li...
1	nice phone, nice up grade from my pantach revu...	4	1	nice phone nice grade pantach revu clean set e...
2	Very pleased	5	1	plea
3	It works good but it goes slow sometimes but i...	4	1	work good goe slow sometim good phone love
4	Great phone to replace my lost phone. The only...	4	1	great phone replac lost phone thing volum butt...

5.1.2 Data Transformation

There were two transformation (feature extraction) techniques that were implemented. TF-IDF and Count Vectorizer. The *max_features* parameter was changed in case of both techniques and the models were tested against various values of that parameter.

5.1.2.1 Term Frequency – Inverse Term Frequency

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf=TfidfVectorizer(min_df=5, max_df=0.95, max_features = 20000,
                      ngram_range = ( 1, 2 ), sublinear_tf = True)
tfidf=tfidf.fit(df['clean_review'])
X=tfidf.transform(df['clean_review'])
y=df['sentiment'].values
```

5.1.2.2 Count Vectorizer

```
from sklearn.feature_extraction.text import CountVectorizer
cv=CountVectorizer(max_features=20000, min_df=5, ngram_range=(1,2))
X=cv.fit_transform(df['clean_review'])
y=df['sentiment'].values
```

5.1.3 Train-Test Ratio

For training, 70% of the dataset was utilized and the remaining 30% was used for testing.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
print(X_train.shape, y_train.shape)
print(X_test.shape, y_test.shape)
```

5.1.4 Machine Learning Algorithms

The following ML algorithms were first implemented by hardcoding on a subset of the dataset to get in-depth understanding of algorithm's working. It was later replaced by the classifier model in the experimentation phase where the entire dataset was used.

5.1.4.1 Naïve Bayes Classifier

Hardcoded Classifier:

```
class NaiveBayes:
```

```

def fit(self, X, y):
    n_samples, n_features = X.shape
    self._classes = np.unique(y)
    n_classes = len(self._classes)

    # calculate mean, var, and prior for each class
    self._mean = np.zeros((n_classes, n_features), dtype=np.float64)
    self._var = np.zeros((n_classes, n_features), dtype=np.float64)
    self._priors = np.zeros(n_classes, dtype=np.float64)

    for idx, c in enumerate(self._classes):
        X_c = X[y == c]
        self._mean[idx, :] = X_c.mean(axis=0)
        self._var[idx, :] = X_c.var(axis=0)
        self._priors[idx] = X_c.shape[0] / float(n_samples)

def predict(self, X):
    X = X.astype('float64')
    y_pred = [self._predict(x) for x in X]
    return np.array(y_pred)

def _predict(self, x):
    posteriors = []

    # calculate posterior probability for each class
    for idx, c in enumerate(self._classes):
        prior = np.log(self._priors[idx])
        posterior = np.sum(np.log(self._pdf(idx, x)))
        posterior = posterior + prior
        posteriors.append(posterior)

    # return class with the highest posterior
    return self._classes[np.argmax(posteriors)]

def _pdf(self, class_idx, x):
    mean = self._mean[class_idx]
    var = self._var[class_idx]
    numerator = np.exp(-((x - mean) ** 2) / (2 * var))
    denominator = np.sqrt(2 * np.pi * var)
    return numerator / denominator

```

Scikit Learn Classifier:

```

from sklearn.naive_bayes import MultinomialNB
alpha_value = 0.0
fit_prior_value = True
class_prior_values = [0.7, 0.3]
model_nb = MultinomialNB(alpha=alpha_value,
                          fit_prior=fit_prior_value, class_prior=class_prior_values)
model_nb.fit(X_train, y_train)

```

5.1.4.2 Support Vector Machine

Hardcoded Classifier:

```

class svm_classifier():
    # initiating the hyperparameters
    def __init__(self, learning_rate, no_of_iterations, _parameter):
        self.learning_rate = learning_rate
        self.no_of_iterations = no_of_iterations
        self.lambda_parameter = lambda_parameter

    # fitting the dataset to SVM Classifier
    def fit(self, X, Y):
        # m --> number of Data points --> number of rows
        # n --> number of input features --> number of columns
        self.m, self.n = X.shape

        # initiating the weight value and bias value
        self.w = np.zeros(self.n)
        self.b = 0
        self.X = X
        self.Y = Y

        # implementing Gradient Descent algorithm for Optimization
        for i in range(self.no_of_iterations):
            self.update_weights()

    # function for updating the weight and bias value
    def update_weights(self):
        # label encoding
        y_label = np.where(self.Y <= 0, -1, 1)

        # gradients ( dw, db)
        for index, x_i in enumerate(self.X):
            condition = ( y_label[index] * (np.dot(x_i, self.w) - self.b) >= 1).any()
            if (condition == True):
                dw = 2 * self.lambda_parameter * self.w
                db = 0
            else:
                dw = 2 * self.lambda_parameter * self.w - np.dot(x_i, y_label[index])
                db = y_label[index]
            self.w = self.w - self.learning_rate * dw
            self.b = self.b - self.learning_rate * db

    # predict the label for a given input value
    def predict(self, X):
        output = np.dot(X, self.w) - self.b
        predicted_labels = np.sign(output)
        y_hat = np.where(predicted_labels <= -1, 0, 1)
        return y_hat

```

Scikit Learn Classifier:

```

from sklearn.svm import LinearSVC
%%time
params = {'C':1, 'penalty': 'l2', 'loss': 'squared_hinge', 'dual': False, 'max_iter':8000}
#creates an instance of the LinearSVC classifier
svm_classifier = LinearSVC(**params)
#trains the LinearSVM classifier on the training data
svm_classifier.fit(X_train, y_train)

```

5.1.4.3 Random Forests**Hardcoded Classifier:****class RandomForest:**

```

def __init__(self, n_trees=50, max_depth=100, min_samples_split=10,
              min_samples_leaf=1, max_features=1000):
    self.n_trees = n_trees
    self.max_depth = max_depth
    self.min_samples_split = min_samples_split
    self.min_samples_leaf = min_samples_leaf
    self.max_features = max_features
    #forest of decision trees
    self.trees = []

def fit(self, X, y):
    self.trees = []
    #create the random forest
    for i in range(self.n_trees):
        #create decision tree
        print(i, end=" ")
        tree = DecisionTreeClassifier( max_depth=self.max_depth, min_samples_split =
                                      self.min_samples_split, min_samples_leaf = self.min_samples_leaf,
                                      max_features=self.max_features)
        #obtain samples from train set
        X_sample, y_sample = self._bootstrap_samples(X,y)
        tree.fit(X_sample, y_sample)
        self.trees.append(tree)
    print()

def predict(self, X):
    #predictions=[[1,0,0..],[1,1,0,..],...] each list is prediction of all samples each tree
    predictions = np.array([tree.predict(X) for tree in self.trees])

    #tree_preds = [[1,1,...],[0,1,..],...] each list represents
    predictions of each sample from all trees
    tree_preds = np.swapaxes(predictions, 0, 1)

    #majority vote for each sample
    predictions = np.array([self._most_common_label(pred) for pred in tree_preds])
    return predictions

```



```

def _bootstrap_samples(self, X, y):
    n_samples = X.shape[0]
    #generate same size sample with duplicates
    idxs = np.random.choice(n_samples, n_samples, replace=True)
    return X[idxs], y[idxs]

def _most_common_label(self, y):
    counter = Counter(y)
    # most common element from [(element, count),...] sorted in descending order
    most_common = counter.most_common(1)[0][0]
    return most_common

def get_params(self, deep=True):
    return {"n_trees": self.n_trees,
            "max_depth": self.max_depth,
            "min_samples_split": self.min_samples_split,
            "max_features": self.max_features}

```

Scikit Learn Classifier:

```

from sklearn.ensemble import RandomForestClassifier
model_rf=RandomForestClassifier(n_estimators=100, criterion='gini', max_depth=None,
                                min_samples_split=2, _samples_leaf=1, max_features='sqrt', verbose=2)
model_rf.fit(X_train,y_train)

```

5.1.4.4 Logistic Regression**Hardcoded Classifier:**

```

def sigmoid(x):
    return 1/(1+np.exp(-x))
class LogisticRegression():
    def __init__(self, lr=0.0001, n_iters=1500):
        self.lr = lr
        self.n_iters = n_iters
        self.weights = None
        self.bias = None

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0
        for _ in range(self.n_iters):
            linear_pred = np.dot(X, self.weights) + self.bias
            predictions = sigmoid(linear_pred)
            dw = (1/n_samples) * np.dot(X.T, (predictions - y))
            db = (1/n_samples) * np.sum(predictions-y)
            self.weights = self.weights - self.lr*dw
            self.bias = self.bias - self.lr*db

    def predict(self, X):
        linear_pred = np.dot(X, self.weights) + self.bias

```

```

y_pred = sigmoid(linear_pred)
class_pred = [0 if y<=0.5 else 1 for y in y_pred]
return class_pred

#model initialization
model_lr=LogisticRegression()

```

Scikit Learn Classifier:

```

from sklearn.linear_model import LogisticRegression as lr
#Model for best param through hypertuning
model_lr = lr(penalty='l2', C=10, solver='newton-cg')
%%time
model_lr.fit(X_train,y_train)

```

5.1.5 Deep Learning Algorithms

The following algorithms were implemented under DL. LSTM was implemented on both cleaned datasets whereas BERT and BERT-BiLSTM models were implemented on the raw dataset.

5.1.5.1 LSTM Model

```

from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LSTM
from keras.layers import Embedding
embedding_layer = Embedding(vocab_size,embedding_vector_length,
                           input_length=max_review_length)
lstm_model = Sequential()
lstm_model.add(embedding_layer)
lstm_model.add(LSTM(units=64, return_sequences=True))
lstm_model.add(Dropout(0.2))
lstm_model.add(LSTM(units=100))
lstm_model.add(Dense(1, activation='sigmoid')) # Output layer
lstm_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

```

5.1.5.2 BERT Model

```

model = BertForSequenceClassification.from_pretrained( "bert-base-cased", num_labels =
2, output_attentions = False, output_hidden_states = False,)

```

Resulting Model Structure:

```

BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(28996, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )

```

```

(encoder): BertEncoder(
  (layer): ModuleList(
    (0-11): 12 x BertLayer(
      (attention): BertAttention(
        (self): BertSelfAttention(
          (query): Linear(in_features=768, out_features=768, bias=True)
          (key): Linear(in_features=768, out_features=768, bias=True)
          (value): Linear(in_features=768, out_features=768, bias=True)
          (dropout): Dropout(p=0.1, inplace=False)
        )
        (output): BertSelfOutput(
          (dense): Linear(in_features=768, out_features=768, bias=True)
          (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
  )
...
)
)
(dropout): Dropout(p=0.1, inplace=False)
(classifier): Linear(in_features=768, out_features=2, bias=True)
)

```

5.1.5.3 BERT-BiLSTM Hybrid Model

```

from torch import nn
from torch.nn.utils.rnn import pack_padded_sequence
class BERTLSTMnn(nn.Module):
    def __init__(self, bert, seq_len, n_layers):
        super().__init__()
        self.bert = bert
        embedding_size = bert.config.to_dict()['hidden_size']
        self.lstm = nn.LSTM(input_size=embedding_size,
            hidden_size=seq_len,
            num_layers=n_layers,
            batch_first=True,
            dropout=0.2,
            bidirectional=True)
        self.dense = nn.Linear(seq_len*2, num_classes)
        self.softmax = nn.Softmax(dim=1)

    def forward(self, sentence):
        with torch.no_grad():
            embedded = self.bert(input_ids=sentence['input_ids'].squeeze(1), attention_mask
                                =sentence['attention_mask'])
            _, (HiddenStates, CellStates) = self.lstm(embedded['last_hidden_state'])
            output = self.dense(torch.cat((HiddenStates[0], HiddenStates[1]), dim = 1))
            output = self.softmax(output)
            return output

```

5.1.6 Evaluation Metrics

To evaluate the model, metrics like accuracy, recall, precision and MCC were calculated by using the sklearn library.

```
y_pred = best_rf.predict(X_test)
mcc = matthews_corrcoef(y_test, y_pred)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("MCC:", mcc)
```

5.2 RESULTS

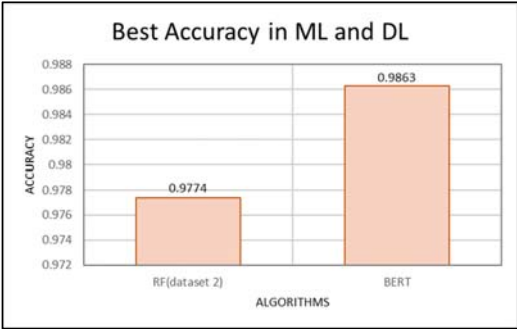
The results of all runs were logged into a google sheet in a systematic format to maintain logs of all runs. The Fig. 5.3 shows a snippet of the structure in which the logs of the algorithms were maintained.

Fig. 5.3 Snippet of result logs for NB

Feature Extraction technique	max features	alpha	fit_prior	class_prior	accuracy	f1 score
TFIDF	20k	1	TRUE	None	0.925	0.95
		1	TRUE	[0.4, 0.6]	0.925	0.949
		1	FALSE	[0.3, 0.7]	0.927	0.951
		0	TRUE	[0.3, 0.7]	0.929	0.952
		0	FALSE	None	0.917	0.943
	15k	1	TRUE	None	0.924	0.949
		1	TRUE	[0.4, 0.6]	0.923	0.948
		1	TRUE	[0.3, 0.7]	0.925	0.95
		1	FALSE	None	0.909	0.936
		0	FALSE	None	0.912	0.939
	10k	1	TRUE	None	0.919	0.947
		1	TRUE	[0.4, 0.6]	0.92	0.945
		1	TRUE	[0.3, 0.7]	0.922	0.948
		1	FALSE	None	0.903	0.933
		1	FALSE	[0.4, 0.6]	0.92	0.945
	5k	1	TRUE	None	0.912	0.942
		1	TRUE	[0.4, 0.6]	0.915	0.942
		1	TRUE	[0.3, 0.7]	0.916	0.944
		1	FALSE	None	0.895	0.927
		1	FALSE	[0.4, 0.6]	0.915	0.942
CountVectoriser	20k	1	FALSE	[0.3, 0.7]	0.923	0.948
		0	TRUE	None	0.923	0.949
		0	TRUE	[0.4, 0.6]	0.924	0.948
		0	TRUE	[0.3, 0.7]	0.924	0.949
		0	FALSE	None	0.919	0.944
	15k	1	FALSE	[0.3, 0.7]	0.921	0.947
		0	TRUE	None	0.922	0.948
		0	TRUE	[0.4, 0.6]	0.922	0.947
		0	TRUE	[0.3, 0.7]	0.922	0.947
		0	FALSE	None	0.916	0.943
	10k	0.5	TRUE	[0.3, 0.7]	0.919	0.946
		0.5	FALSE	None	0.912	0.94
		0.5	FALSE	[0.4, 0.6]	0.919	0.945
		0.5	FALSE	[0.3, 0.7]	0.919	0.946
		0.1	TRUE	None	0.918	0.945
	5k	0.5	FALSE	None	0.906	0.935
		0.5	FALSE	[0.4, 0.6]	0.913	0.942
		0.5	FALSE	[0.3, 0.7]	0.913	0.942
		0.1	TRUE	None	0.913	0.942
		0.1	TRUE	[0.4, 0.6]	0.914	0.942

Furthermore, the findings from the experimentation process are summarized using the graphs and tables given below. Fig. 5.4 shows the highest overall accuracies observed among ML and DL algorithms. RF on dataset 2 from ML and BERT on the raw dataset from DL were the best performing algorithms.

Fig. 5.4 Best Observed Accuracies



Best accuracies achieved by the various ML algorithms over dataset 1 and dataset 2 are depicted in Fig. 5.5 and Fig. 5.6 respectively.

Fig. 5.5 Highest Accuracies by ML algorithms on Dataset 1

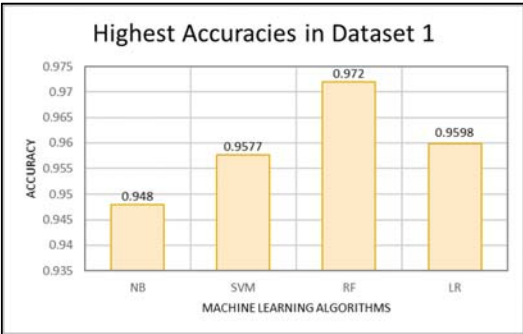


Fig. 5.6 Highest Accuracies by ML algorithms on Dataset 2

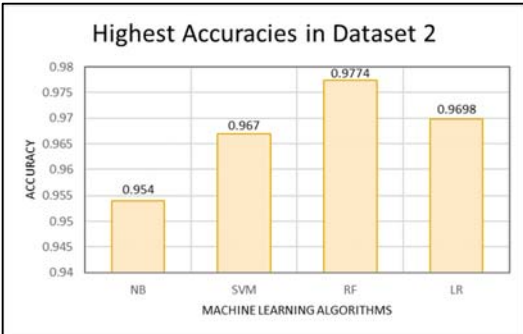
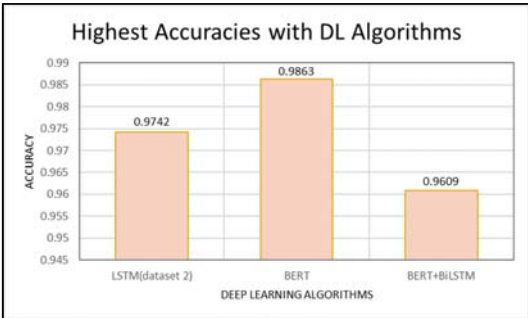


Fig. 5.7 shows the best accuracies achieved by the various DL algorithms overall. BERT is the best performing DL algorithm, followed by LSTM then BERT+BiLSTM hybrid.

Fig. 5.7 Highest Accuracies by DL algorithms



Tables 5.1-5.8 show the performance of various ML algorithms on varying the *max_features* parameter of feature extraction technique. Table 5.9 and Table 5.10 show performance metrics of the DL algorithms.

Table 5.1 Performance of TF-IDF and NB

NB/TFIDF	70:30 split			
	max_features	accuracy	f1-score	mcc score
dataset 1	5k	0.916	0.944	0.776
	10k	0.923	0.948	0.793
	15k	0.926	0.951	0.804
	20k	0.929	0.952	0.81
	300k	0.948	0.965	0.86
dataset 2	max_features	accuracy	f1-score	mcc score
	5k	0.928	0.952	0.81
	10k	0.934	0.956	0.826
	15k	0.938	0.958	0.836
	20k	0.941	0.96	0.844
	300k	0.954	0.97	0.879

Table 5.2 Performance of CV and NB

NB/CV	70:30 split			
	max_features	accuracy	f1-score	mcc score
dataset 1	5k	0.913	0.941	0.774
	10k	0.918	0.945	0.783
	15k	0.921	0.947	0.792
	20k	0.924	0.949	0.799
dataset 2	max_features	accuracy	f1-score	mcc score
	5k	0.924	0.949	0.805
	10k	0.931	0.954	0.821
	15k	0.934	0.955	0.828
	20k	0.936	0.957	0.833

Table 5.3 Performance of TF-IDF and SVM

SVM/TFIDF	70:30 split			
	max_features	accuracy	f1-score	mcc score
dataset 1	5k	0.9394	0.95957	0.838
	10k	0.94604	0.96401	0.8565
	15k	0.9523	0.9681	0.8734
	20k	0.9554	0.9702	0.8818
dataset 2	max_features	accuracy	f1-score	mcc score
	5k	0.95195	0.96785	0.8729
	10k	0.95748	0.97153	0.8877
	15k	0.9622	0.9747	0.9003
	20k	0.9654	0.9768	0.9087

Table 5.4 Performance of CV and SVM

SVM/CV	70:30 split			
	max_features	accuracy	f1-score	mcc score
dataset 1	5k	0.937723	0.95858	0.8336
	10k	0.9485	0.96574	0.8629
	15k	0.9537	0.9691	0.8771
	20k	0.9577	0.9717	0.887
dataset 2	max_features	accuracy	f1-score	mcc score
	5k	0.951764	0.9678	0.8719
	10k	0.95919	0.97271	0.892
	15k	0.9626	0.975	0.9013
	20k	0.967	0.9779	0.913

Table 5.5 Performance of TF-IDF and RF

RF/TFIDF	70:30 split			
	max_features	accuracy	f1-score	mcc score
dataset 1	10k	0.9695	0.9797	0.9189
	20k	0.9698	0.9799	0.9198
	45k	0.9702	0.9802	0.9208
	100k	0.97	0.9801	0.9203
	200k	0.9699	0.98	0.9199
dataset 2	max_features	accuracy	f1-score	mcc score
	10k	0.9774	0.9833	0.9342
	20k	0.9759	0.9839	0.9363
	45k	0.9755	0.9836	0.9353
	100k	0.9766	0.9844	0.9382
	200k	0.97643	0.9842	0.93757

Table 5.6 Performance of CV and RF

RF/CV	70:30 split			
	max_features	accuracy	f1-score	mcc score
dataset 1	5k	0.9714	0.9809	0.92387
	10k	0.97	0.98	0.9202
	15k	0.97011	0.9801	0.9204
	20k	0.97	0.98	0.92
	25k	0.972	0.9814	0.9256
dataset 2	max_features	accuracy	f1-score	mcc score
	5k	0.9752	0.9835	0.9345
	10k	0.9761	0.984	0.9367
	15k	0.9764	0.9843	0.9376
	20k	0.9764	0.9843	0.9376
	25k	0.9769	0.9769	0.9769

Table 5.7 Performance TF-IDF and LR

LR/TFIDF	70:30 split			
	max_features	accuracy	f1-score	mcc score
dataset 1	5k	0.9396	0.9596	0.8399
	10k	0.9461	0.964	0.8573
	15k	0.9543	0.9694	0.879
	20k	0.9581	0.9719	0.8889
dataset 2	max_features	accuracy	f1-score	mcc score
	5k	0.9528	0.9684	0.8748
	10k	0.9593	0.9728	0.8921
	15k	0.9647	0.9764	0.9066
	20k	0.9682	0.9787	0.9159

Table 5.8 Performance CV and LR

LR/CV	70:30 split			
	max_features	accuracy	f1-score	mcc score
dataset 1	5k	0.9375	0.9585	0.8327
	10k	0.9464	0.9642	0.8573
	15k	0.9528	0.9685	0.8745
	20k	0.9598	0.9731	0.8932
dataset 2	max_features	accuracy	f1-score	mcc score
	5k	0.9496	0.9664	0.8655
	10k	0.9595	0.973	0.8925
	15k	0.9249	0.9507	0.7962
	20k	0.9698	0.9798	0.92

Table 5.9 Performance of LSTM Model

	LSTM		
	validation accuracy	test accuracy	MCC score
dataset1	0.9659	0.9665	0.908
dataset2	0.9746	0.9742	0.9322

Table 5.10 Performance of BERT and BERT+BiLSTM Hybrid Model

	validation accuracy	test accuracy	MCC score
BERT	0.9865	0.9863	0.964
BERT+BiLSTM	0.9622	0.9609	0.8989

CHAPTER 6

DESIGN AND INTEGRATION

In this chapter, the application development process is discussed. The chapter covers the React packages and APIs implemented, along with the design concept. This is followed by the backend integration, which covers the data storage and the processing of the application.

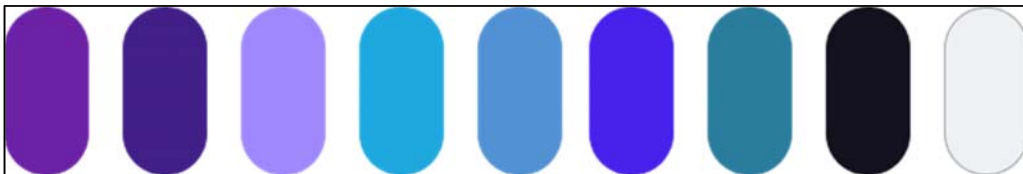
6.1 DESIGN CONCEPT

6.1.1 Color Scheme

With a dark background accented by shades of purple and blue, complemented by #EDF1F4 for text and UI elements, the design creates a cohesive and professional visual identity. This approach enhances visual appeal, facilitates intuitive navigation, and effectively communicates information and data.

- **Background:** The web application is characterised by a prominent dark background colour (#110f1cfc), which establishes a visually striking and dramatic foundation across the entire interface.
- **Text and UI Elements:** #EDF1F4 maintains consistent usage for text and UI elements throughout the application. This choice ensures optimal readability and visual uniformity against the dark background, enhancing user interaction and comprehension.
- **Accent Colors:** Various shades of purple and blue (e.g., #6C22A6, #4921EC, #43208B) are strategically employed as accent colours. These hues add depth to interactive elements and data visualisations, emphasizing key information and enhancing the overall user experience.

Fig. 6.1 Color Palette for Application



6.1.2 Font Scheme

The web application employs two carefully selected fonts to optimize readability and visual appeal. These font choices were strategically made to unify the application's visual identity while prioritizing usability and accessibility, ensuring a seamless user experience. These fonts were imported from the Google fonts library.

- 1 **Nanum Gothic Coding:** Chosen for its clean and legible monospace design, Nanum Gothic Coding ensures clarity in presenting textual content and instructions throughout the

interface. Its consistent spacing enhances readability, making it ideal for maintaining a structured and organized appearance.

Fig. 6.2 Font Style 1: Nanum Gothic Coding



- 2 **Major Mono Display:** Selected for headings and prominent UI elements, Major Mono Display adds a modern and bold aesthetic. This font's distinctive style enhances visual hierarchy, drawing attention to important information and improving overall user engagement.

Fig. 6.3 Font Style 2: Major Mono Display



6.2 REACT PACKAGES USED

1. **React (^18.2.0):** React is the core library for building user interfaces in a declarative and efficient manner. It enables developers to create reusable components and manage application state effectively.
2. **React-dom (^18.2.0):** react-dom provides DOM-specific methods for rendering React components on the web. It integrates seamlessly with React to enable the creation of dynamic and interactive web applications.
3. **Chart.js (^4.4.2):** Chart.js is an open-source JavaScript library that facilitates the creation of visually appealing and interactive charts within web applications. It supports various chart types, such as line, bar, pie, radar, and scatter, enabling developers to choose the most suitable representation for their data.
4. **Chartjs-adapter-moment (^1.0.1):** Chartjs-adapter-moment integrates Moment.js with Chart.js to enhance date and time handling capabilities. This combination allows developers to create sophisticated time-based visualisations with accurate parsing, formatting, and localized date and time support.
5. **jsPDF (^2.5.1):** jsPDF is a popular library for generating PDF documents directly from web pages. It provides a powerful API that enables dynamic creation and downloading of PDFs, making it ideal for exporting charts and other visual content for analysis and sharing.
6. **@iconscout/react-unicons(^1.1.6):** A collection of vector icons for enhancing the user interface with high-quality icons.

6.3 API ENDPOINTS

The endpoints collectively handle various aspects of data submission, sentiment analysis, and data retrieval, providing a robust API for product sentiment analysis. Table 6.2 summarizes the API endpoints utilized in this application and describes them in detail below.

Table 6.1 Summary of API Endpoints

Endpoint	Path	HTTP Method	Description
Form Submit	/form_submit/	POST	Handles the form submission, processes CSV file or product link, predicts sentiment, and generates word cloud.
Dashboard	/dashboard/	GET	Successfully validates dashboard display.
Customer Rating	/customer_rating/	GET	Provides customer rating distribution.
Aspect Modelling	/aspect_modelling/	GET	Provides analysis of negative aspects from reviews.
Customer Overtime Positive Sentiment	/customer_overtime_positive_sentiment/	GET	Provides positive sentiment data over time.
Customer Overtime Sentiment	/customer_overtime_sentiment/	GET	Provides sentiment data (positive and negative) over time.
Sentiment Prediction	/sentiment_prediction/	GET	Provides the count of reviews by sentiment prediction (positive/negative).
Sentiment Info	/sentiment_info/	GET	Provides sentiment prediction data and reviews data for the last month.

1. Form Submit (/form_submit/):

- **Description:** These endpoints handle form submissions where a user can upload a CSV file or provide a product link. It processes the data, updates the database, performs sentiment analysis, and generates word clouds.
- **HTTP Method:** POST
- **Return Type:** JsonResponse with a success message.

2. Dashboard (/dashboard/):

- **Description:** Successfully validates dashboard display.
- **HTTP Method:** GET
- **Return Type:** JsonResponse with a message indicating the dashboard display.

3. Customer Rating (/customer_rating/):

- **Description:** Provides a distribution of customer ratings from the reviews.
- **HTTP Method:** GET
- **Return Type:** JsonResponse containing rating data.

4. Aspect Modelling (/aspect_modelling/):

- **Description:** Analyses and provides negative aspects extracted from the reviews.
- **HTTP Method:** GET
- **Return Type:** JsonResponse with negative aspects.

5. Customer Overtime Positive Sentiment (/customer_overtime_positive_sentiment/):

- **Description:** Provides the count of positive sentiment reviews over time, broken down by month and year.
- **HTTP Method:** GET
- **Return Type:** JsonResponse with positive sentiment data over time.

6. Customer Overtime Sentiment (/customer_overtime_sentiment/):

- **Description:** Provides the count of both positive and negative sentiment reviews over time, broken down by month and year.
- **HTTP Method:** GET
- **Return Type:** JsonResponse with sentiment data over time.

7. Sentiment Prediction (/sentiment_prediction/):

- **Description:** Provides the count of reviews classified as positive or negative.
- **HTTP Method:** GET
- **Return Type:** JsonResponse with prediction data.

8. Sentiment Info (/sentiment_info/):

- **Description:** Provides overall sentiment prediction data and details of positive and negative reviews for the last month.
- **HTTP Method:** GET
- **Return Type:** JsonResponse with prediction data and last month's review data.

6.4 DASHBOARD ELEMENTS

The dashboard provides a comprehensive visualization of product performance and customer sentiment. This collection of visualisations enables real-time, interactive analysis of customer feedback, helping both businesses and consumers make data-driven decisions. It includes:

- **Line Chart:** Displays monthly product performance trends based on customer sentiment predictions.
- **Pie Chart:** Shows the distribution of customer ratings from the dataset.
- **Doughnut Chart:** Illustrates the proportion of positive and negative sentiment score predictions.
- **Stacked Bar Chart:** Offers a detailed view of monthly sentiment ratios, enhancing the line chart's trend overview.

- **Polar Area Chart:** Categorizes feedback by topics like battery, display, and hardware, identifying sentiment for each—useful for spotting improvement areas and making informed purchase decisions.
- **Word Clouds:** Visualize frequently mentioned words in reviews, providing quick insights into common themes and topics.

6.5 BACKEND PROCESSING

6.5.1 Data Storage

Django web applications access and manage data through Python objects referred to as models. Models define the structure of stored data, including the field types and possibly also their maximum size, default values, selection list options, help text for documentation, label text for forms, etc. In this application, the Review model corresponds to a table where each row represents a customer reviews with attributes for the review text, associated rating, and the date it was created. It also includes fields for the cleaned version of the review text and the predicted sentiment, facilitating the analysis of customer feedback by storing both raw and processed data. It consists of the following attributes:

1. **created_at:** This column stores the date when the review was created. It can be null if the date is not provided.
2. **review_text:** This column contains the original text of the review. It is required and cannot be null.
3. **cleaned_review_text:** This column contains a cleaned version of the review text, which may be used for preprocessing before sentiment analysis. This field is optional and can be null or blank.
4. **rating:** This column stores the rating associated with the review. It is a floating-point number and is required.
5. **sentiment_prediction:** This column stores the sentiment prediction for the review. It is an integer that typically indicates different sentiments (e.g., 0 for negative and 1 for positive). This field is optional and can be null until the sentiment is predicted.

Table 6.2 Review Table in Database

Attribute	Data Type
created_at	Date
review_text	Text
cleaned_review_text	Text
rating	Float
sentiment_prediction	Integer

6.5.2 Data Collection

Users can upload CSV files containing reviews or provide product links to scrape reviews. When a CSV file is uploaded, the reviews are read from the file and stored in the Review model. When a product link is provided, the application scrapes reviews from the product page and stores them in the Review model.

6.5.3 Data Processing

- **Cleaning:** The `cleaned_review_text` field may be used to store a processed version of the `review_text` where unnecessary characters or words are removed.
- **Sentiment Analysis:** The `sentiment_prediction` field is populated with the results of the sentiment analysis, indicating whether the review is positive or negative.

6.5.4 Data Retrieval and Analysis

The data is retrieved from the database using Django's ORM (Object-Relational Mapping), which translates Django model queries into SQL queries. Various endpoints allow retrieval of data for analysis and visualisation. A few examples include:

- **get_sentiment_prediction:** Returns the count of reviews by sentiment prediction.
- **get_customer_rating:** Provides a distribution of customer ratings.
- **get_customer_overtime_sentiment:** Shows sentiment trends over time.

CHAPTER 7

IMPLEMENTATION

This part will cover the important code snippets from the application implementation like getting input from user, web-scraping process, sentiment analysis process, dashboard elements, etc.

7.1 FRONT-END CODES

1. Home page: home.jsx

```
const Home = () => {
  const navigate = useNavigate();
  const [uploadedFile, setUploadedFile] = useState(null);
  const handleFileChange = (event) => {
    const file = event.target.files[0];
    setUploadedFile(file);
  };
  const handleSubmit = async (event) => {
    event.preventDefault();
    const formData = new FormData();
    formData.append("myfile", uploadedFile);
    formData.append("product_link", event.target.product_link.value);
    const csrftoken = getCookie("csrftoken");
    try {
      const response = await fetch("http://localhost:8000/form_submit/", {
        credentials: "include",
        method: "POST",
        headers: {
          Accept: "application/json",
          "X-CSRFToken": csrftoken,
        },
        body: formData,
      });
    } catch (error) {
      console.error("Error submitting form:", error);
    }
  };
  return (
    <div className="text">Upload CSV File</div>
```

```

    <form
      method="post"
      encType="multipart/form-data"
      onSubmit={handleSubmit} >
      <CSRFToken />
      <div className="upload-csv">
        <label htmlFor="myfile" className="upload-button">
          <UiUploadAlt className="upload-icon" />
          <input id="myfile" type="file" name="myfile"
onChange={handleFileChange} />
        </label>
        {uploadedFile && (
          <span className="uploaded-file">{uploadedFile.name}</span>
        )}
      </div>
      <div className="text text-center">OR</div>{" "}
      <div className="text">Input Product URL</div>{" "}
      <input type="text" name="product_link" placeholder="example:
https://www.amazon.in/dp/B07FQ4Q7MB" />
      <div className="submit">
        <button type="submit" className="submit-button">
          Submit
        </button>
      </div>
    </form>);
  };
  export default Home;

```

2. Word Cloud Visualization:

```

import React, { useState, useEffect } from 'react';
const WordCloud = ({ imageName }) => {
  const [imageSrc, setImageSrc] = useState("");
  useEffect(() => {
    fetch(`http://localhost:8000/media/${imageName}.png`)
      .then(response => response.blob())
      .then(blob => {
        const imageUrl = URL.createObjectURL(blob);
        setImageSrc(imageUrl); })
      .catch(error => {
        console.error('Error fetching WordCloud image:', error); });
  }, []);
  return (
    <div className="wordcloud-container"> { /* Add a container div */}
      {imageSrc && (
        <img className="wordcloud-image" src={imageSrc} alt="WordCloud" />
      )}
    </div> );
  };
  export default WordCloud;

```

3. Graph Visualization: LineGraph.jsx

```

const LineGraph = () => {
  useEffect(() => {
    fetchData();
  }, []);
  const fetchData = async () => {
    try {
      const response = await fetch(
        "http://localhost:8000/customer_overtime_sentiment/"
      );
      const responseData = await response.json();
      const zeroData = responseData.zero_overtime_sentiment_data;
      const oneData = responseData.one_overtime_sentiment_data;
      const labels = [
        ...new Set([...Object.keys(zeroData), ...Object.keys(oneData)]),
      ];
      labels.sort((a, b) => new Date(a) - new Date(b));

      const zeroValues = [];
      const oneValues = [];

      labels.forEach((label) => {
        zeroValues.push(zeroData[label] || 0);
        oneValues.push(oneData[label] || 0);
      });

      const ctx = chartRef.current.getContext("2d");
      const ctx2 = chartRef.current.getContext("2d");
      const gradient1 = ctx2.createLinearGradient(0, 0, 0, 400);
      const gradient2 = ctx2.createLinearGradient(0, 0, 0, 400);
      lineChartRef.current = new Chart(ctx, {
        type: "line",
        data: {
          labels: labels,
          datasets: [
            {
              label: " Positive",
              data: oneValues,
            },
            {
              label: "Negative",
              data: zeroValues,
            },
          ],
        },
        options: {
          scales: {
            y: {
              beginAtZero: true,
              title: {
                display: true,
                text: "Count Of Reviews",
              },
            },
          },
        },
      });
    } catch (error) {
      console.log(error);
    }
  };
};

```

```

      x: {
        type: "time",
        title: {
          display: true,
          text: "Time",
        },
        time: {
          displayFormats: {
            quarter: "MMM YYYY",
          }},
        plugins: [ChartDataLabels],
      });
    };
  } catch (error) {
    console.error("Error fetching data:", error);
  }
};
return <canvas ref={chartRef} />;
};
export default LineGraph;

```

4. Main Content: (MainContent.jsx)

```

const MainContent = () => {
  return (
    <div className="main-content">
      <div className="chart-layout">
        <div className="chart-element line-graph">
          <h5>Product Performance Overtime </h5>
          <LineGraph />
        </div>
        <div className="chart-element card1">
          <CardOne />
        </div>
        <div className="chart-element pie-chart">
          <h5>Customer Ratings</h5>
          <PieChart />
        </div>
      <div className="chart-element card2">
        <CardTwo />
      </div>

      <div className="chart-element doughnut-chart ">
        <h5>Sentiment Prediction Score</h5>
        <DoughnutChart />
      </div>

      <div className="chart-element stacked-bar">
        <h5>Ratio of Positive and Negative Sentiments Overtime</h5>
        <StackedBarChart />
      </div>
    </div>
  );
};

```



```

        <div className="chart-element chart-image">
        </div>

        <div className="chart-element polar-chart">
            <h5>Top Issues</h5>
            <PolarAreaChart />
        </div>
        <div className="chart-element word-cloud">
            <h5>Positive Sentiment Word Cloud</h5>
            <WordCloud imageName="wordcloud_positive" />
        </div>
        <div className="chart-element word-cloud">
            <h5>Negative Sentiment Word Cloud</h5>
            <WordCloud imageName="wordcloud_negative" />
        </div>
    </div>
    </div>
    );
};
export default MainContent;

```

7.2 BACK-END CODES

1. Database schema definition: (models.py)

```
from django.db import models
```

```

class Review(models.Model):
    created_at = models.DateField(null=True)
    review_text = models.TextField()
    cleaned_review_text = models.TextField(null=True, blank=True)
    rating = models.FloatField()
    sentiment_prediction = models.IntegerField(null=True, blank=True)

```

2. Deep learning model integration: (apps.py)

```
from transformers import BertTokenizer, BertForSequenceClassification
```

```

class SentimentAnalyserConfig(AppConfig):
    # Load BERT model and tokenizer
    model_path = os.path.join(settings.BASE_DIR, 'app', 'models', 'model')
    tokeniser_path = os.path.join(settings.BASE_DIR, 'app', 'models', 'tokeniser')
    tokenizer = BertTokenizer.from_pretrained(tokeniser_path)
    model = BertForSequenceClassification.from_pretrained(model_path)

    name = 'app'

```

3. Form submit API: (views.py)

```

def form_submit(request):
    if request.method == 'POST':
        Review.objects.all().delete()

```

```

if 'myfile' in request.FILES:
    review_data = request.FILES['myfile']

    if not review_data.name.endswith('csv'):
        messages.info(request, 'Please upload CSV files only.')
        return render(request, 'home.html')
    else:
        messages.info(request, 'File successfully uploaded.')

    data = review_data.read().decode('UTF-8')
    io_string = io.StringIO(data)
    next(io_string)
    for column in csv.reader(io_string, delimiter=',', quotechar='"'):
        created = Review.objects.update_or_create(
            created_at = column[0],
            review_text = column[1],
            rating = column[2]
        )
    elif 'product_link' in request.POST:
        product_link = request.POST['product_link']
        try:
            review_data = process_product_link(product_link)

            for index, row in review_data.iterrows():
                Review.objects.update_or_create(
                    created_at=row['Date'],
                    review_text=row['Review'],
                    rating=row['Rating']
                )
        except Exception as e:
            messages.error(request, 'Error fetching data from the provided link:')
            return render(request, 'home.html')
    predict_sentiment()
    IMAGE_PATH = generate_wordcloud()
    return JsonResponse({'message': 'Form submitted successfully'})

return render(request, 'home.html')

```

4. Sentiment analysis functionality: (utils.py)

```

def predict_sentiment():
    reviews = Review.objects.all()
    predictions = []
    probabilities = []
    true_labels = []

    for review in reviews:
        inputs = SentimentAnalyserConfig.tokenizer(review.review_text, return_tensors='pt',
            truncation=True, padding=True, max_length=380) #change maxlen?
        with torch.no_grad():
            outputs = SentimentAnalyserConfig.model(**inputs)

```

```

logits = outputs.logits
prediction = torch.argmax(logits, dim=1).item()
proba = torch.softmax(logits, dim=1).max().item()
probabilities.append(round(proba * 100, 2))
predictions.append(prediction)
review.sentiment_prediction = prediction
review.save()
return probabilities, predictions

```

5. Amazon web scraping functionality: (utils.py)

```

def get_amazon_search(url, driver):
    try:
        driver.get(url)
        return driver.page_source
    except Exception as e:
        print(f'Error: {e}')
        return None

def parse_review_data(page, driver):
    soup = BeautifulSoup(page, "html.parser")
    link = soup.find("a", {'data-hook': "see-all-reviews-link-foot"})['href']

    reviews = []
    dates = []
    ratings = []

    for page_num in range(20):
        page_source =
get_amazon_search('https://www.amazon.in'+link+'&pageNumber='+str(page_num),
driver)
        soup = BeautifulSoup(page_source, features='lxml')

        for review in soup.findAll("div", {'data-hook': "review"}):
            review_text = review.find("span", {'data-hook': 'review-body'}).text.strip()
            review_date = review.find("span", {'data-hook': 'review-date'}).text.strip().split(' on
')[1]
            review_date = datetime.strptime(review_date, '%d %B %Y').strftime('%Y-%m-%d')
            review_rating = float(review.find("span", {'class': 'a-icon-alt'}).text.split()[0])

            reviews.append(review_text)
            dates.append(review_date)
            ratings.append(review_rating)

    review_data = pd.DataFrame({'Date': dates, 'Review': reviews, 'Rating': ratings})
    if not review_data.empty:
        return review_data
    else:
        print('ERROR: Failed to retrieve reviews.')
        return None

```

```
def process_product_link(url):
    service = Service(CHROMEDRIVER_PATH)
    driver = webdriver.Chrome(service=service)
    driver.set_page_load_timeout(500)

    login_amazon(driver)
    page=get_amazon_search(url, driver)

    if page:
        review_data = parse_review_data(page, driver)
        if review_data is not None:
            scraper_dir = os.path.join(settings.BASE_DIR, 'app', 'webscraper')
            os.makedirs(scraper_dir, exist_ok=True)
            scraper_path = os.path.join(scraper_dir, 'AmazonReviews.csv')
            review_data.to_csv(scraper_path, index=False)
            print("Data saved to 'AmazonReviews.csv'")
            return review_data
        else:
            print("No review data found.")
    else:
        print("Failed to fetch search results.")

    driver.quit()
```

CHAPTER 8

OUTPUT

This chapter covers the images of the resulting application interface. Images of the landing page, the functionality to provide input through URL or CSV files, the resulting dashboard after sentiment analysis, downloaded report have been shown below.

Fig. 8.1 CSV file given as input on Home Page

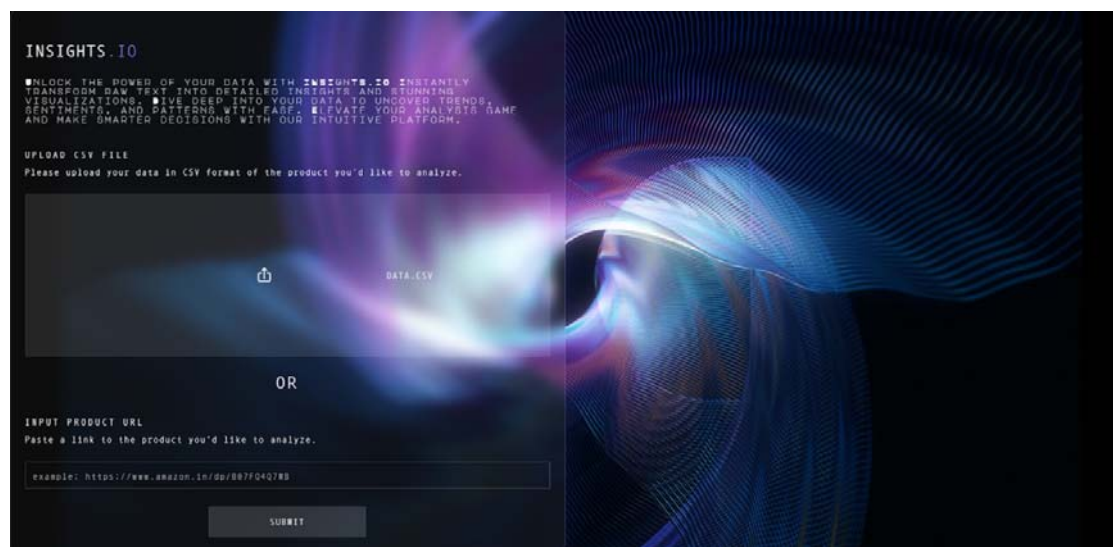


Fig. 8.2 URL given as input on Home Page

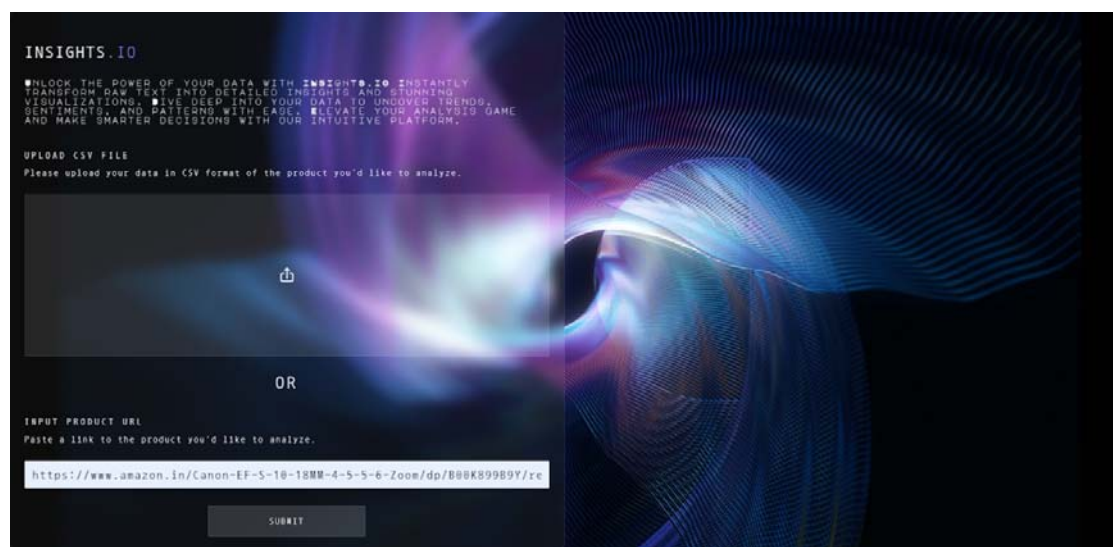


Fig. 8.3 Dashboard

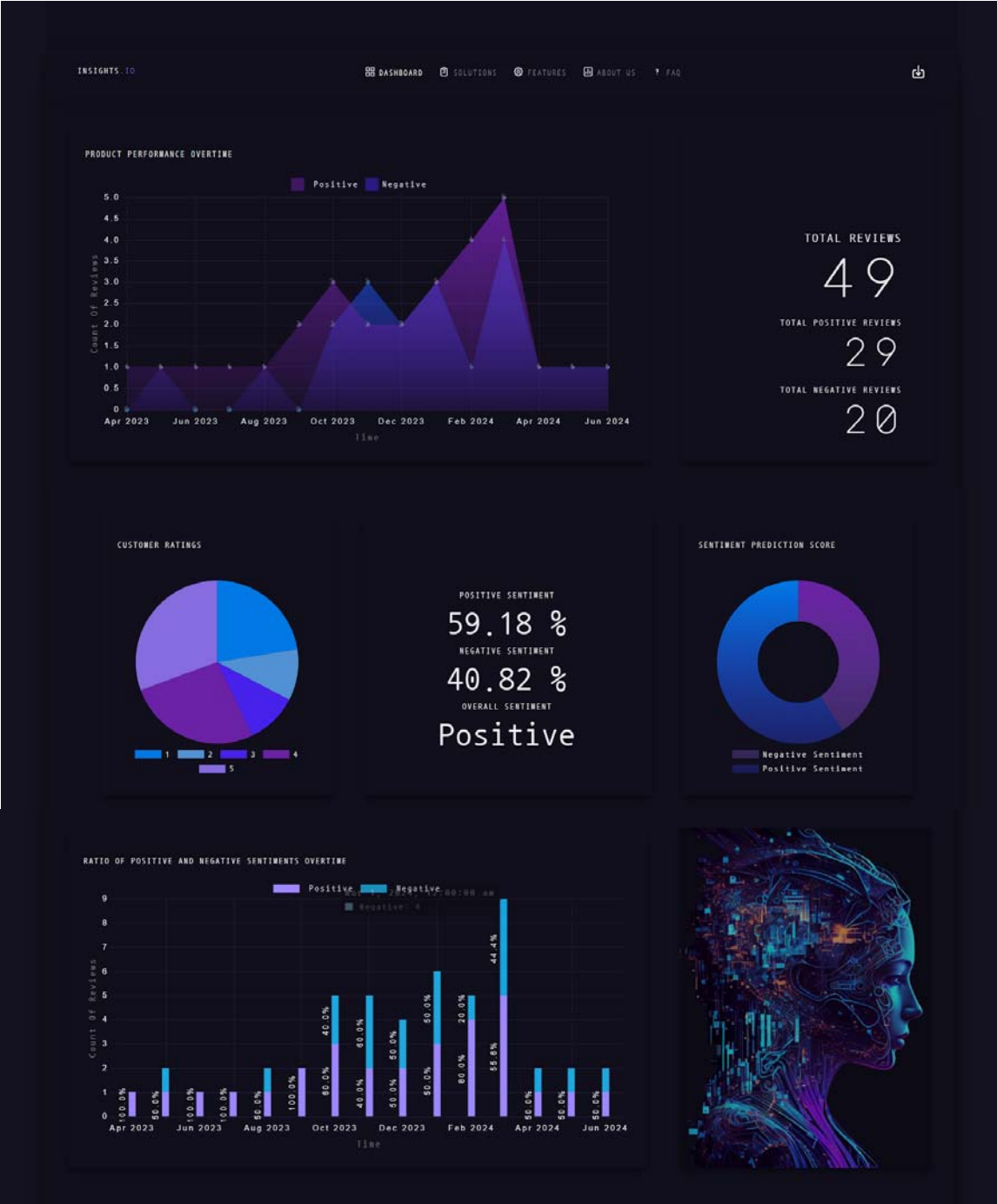
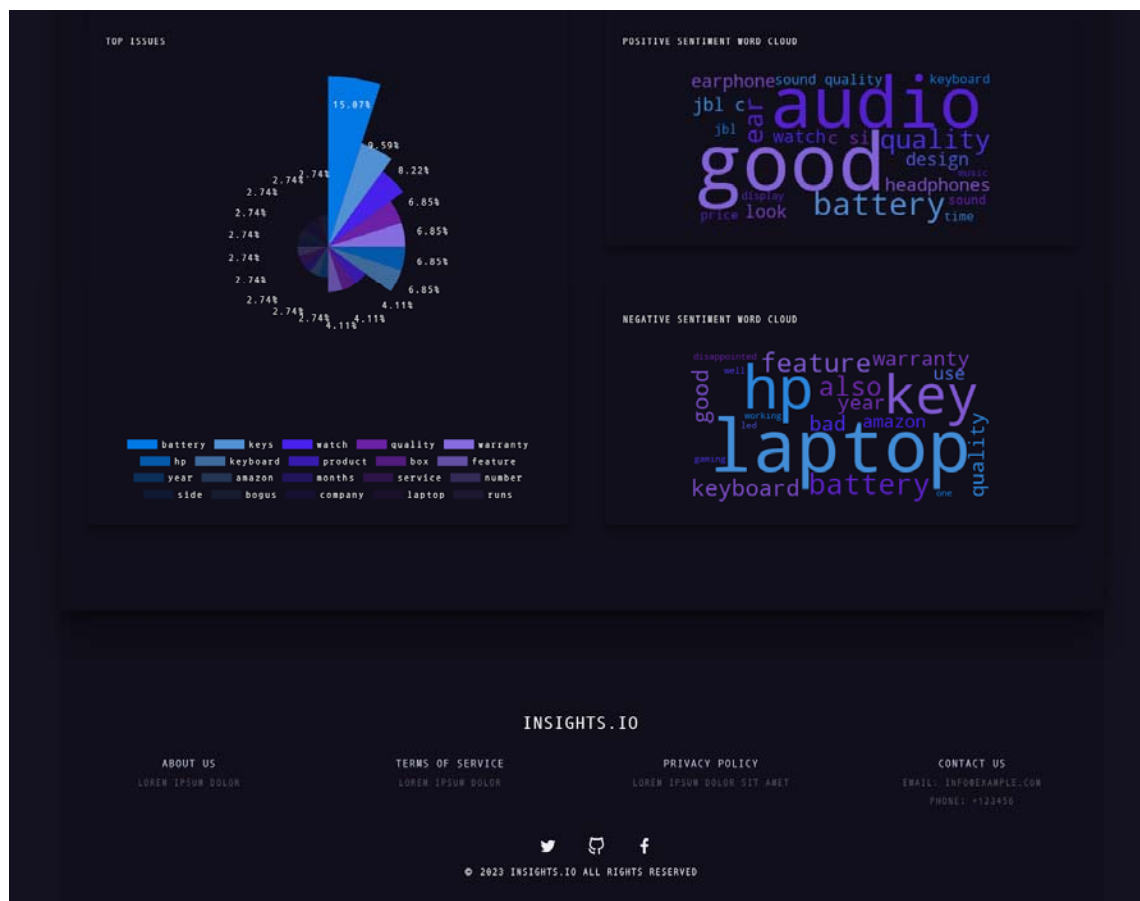


Fig. 8.4 Dashboard (contd.)



CONCLUSION

In today's digital age, product review sentiment analysis has become essential for businesses to understand customer feedback and refine their strategies. This research delved into various machine learning (ML) and deep learning (DL) techniques for sentiment analysis, providing valuable insights into their performance. A comparative study was conducted, which revealed that the Random Forest (RF) algorithm performed best among ML models with an accuracy of 0.9774, while the Bidirectional Encoder Representations from Transformers (BERT) model excelled among DL models with an accuracy of 0.9863.

This study highlighted the effectiveness of different algorithms and provided a comprehensive understanding of the methodologies and techniques involved in sentiment analysis. The experimentation process demonstrated the superiority of specific combinations of feature extraction methods and algorithms, with RF and BERT standing out in their respective categories.

The developed application leveraged the insights gained from the research, enabling users to input data via URL or CSV and generate a detailed sentiment analysis dashboard. This tool empowers businesses and consumers to make data-driven decisions, enhancing overall business performance and improving customer satisfaction.

FUTURE SCOPE

The product reviews have a wide and encouraging potential for the future in which they can be used to analyze sentiments. As businesses increasingly prioritize consumer feedback, it is important that they improve on their sentiment analysis techniques. More robustness and accuracy of the model may be achieved by expanding the dataset to encompass a wider range of reviews from multiple sources. Moreover, more in-depth study of advanced deep-learning models would further improve sentiment classification accuracy.

Combining sentiment analysis with real-time data processing abilities will ensure timely responses and prompt action by firms as they obtain immediate insights into customer sentiments. Additionally, it will enable customers to briefly look at the product before buying it. Moreover, integrating multilingual sentiment analysis can enhance usability of the application globally since this caters for different audience bases across continents. Overall user experience can also be improved through better user interface design as well as interactivity.

Lastly, other natural language processing tasks like topic modeling and emotion detection, sentiment analysis could be implemented to enable marketers to fully understand customers' feeling about products or services offered by companies

REFERENCES

1. Coyne, E., Smit, J., Güner, L., "Sentiment analysis for Amazon.com reviews," ResearchGate.
2. Salem, M. A., & Maghari, A. Y., "Sentiment analysis of mobile phone products reviews using classification algorithms," In 2020 International conference on promising electronic technologies (ICPET), IEEE, 2020, pp. 84-88.
3. Putri, D. A., Kristiyanti, D. A., Indrayuni, E., Nurhadi, A., & Hadinata, D. R., "Comparison of naive bayes algorithm and support vector machine using pso feature selection for sentiment analysis on e-wallet review," In Journal of Physics: Conference Series, Vol. 1641, No. 1, IOP Publishing, 2020, p. 012085.
4. Abraham, M. P., & KR, U.K.R., "Feature based sentiment analysis of mobile product reviews using machine learning techniques," International Journal, Vol. 9, No. 2, 2020.
5. Fang, X., & Zhan, J., "Sentiment analysis using product review data," Journal of Big Data, Vol. 2, No. 1, 2015, pp. 1-14.
6. Kanakaraj, M., & Guddeti, R. M. R., "NLP based sentiment analysis on Twitter data using ensemble classifiers," In 2015 3Rd international conference on signal processing, communication and networking (ICSCN), IEEE, March 2015, pp. 1-5.
7. Wankhade, M., Rao, A. C. S., & Kulkarni, C., "A survey on sentiment analysis methods, applications, and challenges," Artificial Intelligence Review, Vol. 55, No. 7, 2022, pp. 5731-5780.
8. Pati, S. P., & Pradhan, B. B., "Comparison between Machine Learning Algorithms Used for Sentiment Analysis," International Journal of Advanced Research in Engineering and Technology (IJARET).
9. Nandwani, P., & Verma, R., "A review on sentiment analysis and emotion detection from text," Social Network Analysis and Mining, Vol. 11, No. 1, 2021, p. 81.
10. Rathee, N., Joshi, N., & Kaur, J., "Sentiment analysis using machine learning techniques on Python," In 2018 Second International Conference on Intelligent Computing and Control Systems (ICICCS), IEEE, June 2018, pp. 779-785.
11. Bansal, B., & Srivastava, S., "Sentiment classification of online consumer reviews using word vector representations," Procedia computer science, Vol. 132, 2018, pp. 1147-1153.
12. Salinca, A., "Business reviews classification using sentiment analysis," In 2015 17th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC), IEEE, September 2015, pp. 247-250.
13. Ahmed, S., & Danti, A., "Effective sentimental analysis and opinion mining of web reviews using rule based classifiers," In Computational Intelligence in Data Mining—Volume 1: Proceedings of the International Conference on CIDM, Springer India, December 2015, pp. 171-179.
14. Hirsch, S., Novgorodov, S., Guy, I., & Nus, A., "Generating tips from product reviews," In Proceedings of the 14th ACM International Conference on Web Search and Data Mining, 2021, pp. 310-318.
15. Kumar, K. S., Desai, J., & Majumdar, J., "Opinion mining and sentiment analysis on online customer review," In 2016 IEEE International Conference on Computational Intelligence and Computing Research (ICCIC), IEEE, December 2016, pp. 1-4.
16. Sentiment Analysis Logs. Available: <https://docs.google.com/spreadsheets/d/1aQkursUWirjdzWigB2hT6npgc4h7-WuWmF5RyIRknoE/edit?usp=sharing>
17. ReactJS Documentation: <https://legacy.reactjs.org/docs/getting-started.html>

18. React Tutorial: <https://react.dev/learn>
19. Django Documentation: <https://docs.djangoproject.com/en/5.0/>
20. ChartJS Documentation: <https://www.chartjs.org/docs/latest/>
21. jsPDF Documentation: <https://artskydj.github.io/jsPDF/docs/jsPDF.html>
22. reactunicons Documentation: <https://www.npmjs.com/package/@iconscout/react-unicons>
23. Tensorflow Documentation: https://www.tensorflow.org/api_docs
24. Scikit-learn Documentation: <https://scikit-learn.org/0.21/documentation.html>
25. Django CSRF Protection: <https://www.techiediaries.com/django-react-forms-csrf-axios/>
26. Django SQLite Database: <https://medium.com/@codewithbushra/using-sqlite-as-a-database-backend-in-django-projects-code-with-bushra-d23e3100686e>
27. BERT for Sequence Classification Tutorial: <https://medium.com/@alexmriggio/bert-for-sequence-classification-from-scratch-code-and-theory-fb88053800fa>
28. Illustrated Transformers: <https://jalammar.github.io/illustrated-transformer/>
29. Illustrated BERT: <https://jalammar.github.io/illustrated-bert/>
30. Google Fonts: <https://fonts.google.com/>